

Audience: LEWG, SG14, WG21

Document number: P0447R24

Date: 2023-10-26

Project: Introduction of std::hive to the standard library

Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

INTRODUCTION OF STD::HIVE TO THE STANDARD LIBRARY

Table of Contents

- I. [Introduction](#)
- II. [Questions for the committee](#)
- III. [Motivation and Scope](#)
- IV. [Impact On the Standard](#)
- V. [Design Decisions](#)
- VI. [Technical Specification](#)
- VII. [Acknowledgments](#)
- VIII. Appendices:
 - A. [Basic usage examples](#)
 - B. [Reference implementation benchmarks](#)
 - C. [Frequently Asked Questions](#)
 - D. [Specific responses to previous committee feedback](#)
 - E. [Typical game engine requirements](#)
 - F. [Time complexity requirement explanations](#)
 - G. [Original reference implementation differences and link](#)
 - H. [User experience reports](#)

- I. [Brief guide for selecting an appropriate container based on usage and performance](#)
- J. [Hive constraints summary](#)
- K. [Links to prior art](#)
- L. [Further info on non-reference-implementation designs](#)

Revision history

- R24: Corrected HTML errors (some highlighting incorrect in tech specification, other places). Overhauled info for supporting small types without overalignment, in alt implementation details (Appendix L). Minor corrections to tech spec. Slight rewording of shrink_to_fit tech spec to be closer in wording to vector's.
- R23: Correction/update to constexpr usage section in appendices. Correction of bit_cast to reinterpret_cast in Design Decisions. Added section within Design Decisions->Erased-element location recording mechanism, detailing the various approaches possible for keeping track of which blocks contain erasures. Correction to Design Decisions->Collection of element memory blocks + metadata. Addition of explanatory infographic to intro, courtesy of Victor Reverdy's suggestion. $\Leftrightarrow$ exclusion note removed from tech overview. Growth factor "greater than 1" removed from overview and "which need not be integral" added to bring in line with other mentions of growth factor in Standard. Note1 'poem' removal in hive.overview taken out, referred to LWG directly for review. LWG feedback on time complexity wording: make maximum required by current implementation, if at later point it needs to be adjusted it can, future ABI breakage is more a concern than time complexity. Time complexity for erasure-handling removed as it is possible to make it $O(1)$ for all types without overaligning small types (see end of appendix L for detail). Tim Song and Jens Maurer signed off on erasure-handling time complexity wording ($x + y$ where x = in elements, y = in element blocks, this approach is mirrored in parts of the standard). Further updates based on Ben Craig's feedback & private review group feedback. Other corrections to time complexity. Updates to alt implementation details (Appendix L). Removed clear() description from tech spec as is covered by the containers blanket wording and the overview blanket wording. Erase descriptions also reduced as portions were covered by overview blanket wording.
- R22: Addition of Hive constraints summary in appendices. Addition of prior art info to appendices. Additional information around alternative (vector-of-pointer based) implementations added in Appendices and said information in Design Decisions modified. Some other appendix items updated.
- R21: Included note in Design Decisions section regarding conditions under which block capacity limits are copied between hives, and formalized this in the Technical Specification. Corrections to Appendix F. Correction to title in Design Decisions section.
- R20: Removal of $==$ $!=$ and $\Leftrightarrow$ container operators. Reasoning for this added to the [appendix](#), under the FAQ category. Addition of reference implementation licensing compatibility with other licensing to appendix D. Minor corrections. Removal of complexity specification for sort() to allow for different algorithms to be used. Iterator invalidation information moved into tech spec function descriptions. Tech spec overhaul via Ben Craig's feedback. C++20 ranges overloads

added. Tech spec numbering removed, replaced with tags. Addition of 'block_capacity_hard_limits' function. More FAQ entries. Remove priority template parameter and reasoning added to FAQ. Tech spec overhaul via Jonathan Wakely's feedback. Removal of memory() (see FAQ), trim() renamed to trim_capacity(). Conditions for functions with block limits as an argument changed from throwing when not satisfying requirements, to undefined behavior (now that requirements can always be satisfied by user via calling block_capacity_hard_limits()). Addition of unique() functions since optimal implementation is non-intuitive. Clarification of erase() iterator invalidation rules. iterator get_iterator(pointer p) changed to iterator get_iterator(const_pointer p). Corrections to synopsis. Removal of advance/distance/next/prev overloads from tech spec (this allows them to be specialisations within those functions for implementors). Removed copy constructors and operator= from [hive.cons] as these are covered in [sequence.req] (this is reflected in other sequence container [cons]). hive_limits constructor changed to constexpr to allow for constexpr calling from block_capacity_hard_limits(). is_active(const_iterator) added. reverse_iterator and const_reverse_iterator now equivalent to std::reverse_iterator<iterator/const_iterator> in the tech spec. Corrections/additions to time complexity details in tech spec. Tech spec update based on Tim Song's feedback and committee feedback. trim_capacity(n) overload added. hive_limit constructor defaults changed to separate overloads.

- R19: Correction to intro. Addition of sort() to invalidation rules. Removal of questions for the committee based on Gašper Ažman's feedback. Minor corrections. Moved constexpr explanation to appendix D. Addition and removal of questions from appendix D.
- R18: Addition of <=> operator. Addition of basic guide for container selection within/without the standard library, in appendix. Addition of edits to [containers.general] and [sequence.reqmts] in technical specification. Update 22.3.14.1. Some rewording.
- R17: Addition of appendix containing reported user experiences. Editing of constexpr exploration.
- R16: Explanation of desired clear() behavior added in Design Decisions section. References to colony reference implementation changed to refer to hive reference implementation (C++20 only). Textual corrections. Range-constructor corrected to allow sentinels.
- R15: Added throw details to splice. Further design decisions information on reshape and splice. Assign() overload for sentinels (differing iterator types) added. Minor text snafu corrections. Colony changed to hive based on D2332R0.
- R14: get_iterator_from_pointer changed to get_iterator - the pointer part is implied by the fact that it's the only argument. Added const_iterator overload for get_iterator - which takes a const_pointer and is a const function. Some wording corrections, additional design decisions information. HTML corrections.
- R13: Revisions based on committee feedback. Skipfield template parameter changed to priority enum in order to not over-specify container implementation. Other wording changes to reduce over-specifying implementation. Some non-member template functions moved to be friend functions. std::limits changed to std::colony_limits. block_limits() changed to block_capacity_limits().
- R12: Fill, range and initializer_list inserts changed to void return, since the insertions are not guaranteed to be sequential in terms of colony order and therefore returning an iterator to the first insertion is not useful. Non-default-value fill constructor changed to non-explicit to match other std:: containers. Correction to reserve() wording. Other minor corrections and clarity improvements.

- R11: Overhaul of technical specification to be more 'wording-like'. Minor alterations & clarifications. Additional alternative approach added to Design Decisions under skipfield information. Overall rewording. Reordering based on feedback. Removal of some easily-replicated 'helper' functions. Change to noexcept guarantees. Assign added. `get_block_capacity_limits` and `set_block_capacity_limits` functions renamed to `block_limits` and `reshape`. Addition of block-limits default constructors. `Reserve()` and `shrink_to_fit()` reintroduced. `trim()`, `erase` and `erase_if` overloads added.
- R10: Additional information about time complexity requirements added to appendix, some minor corrections to time complexity info. The 'bentley pattern' (this was always a temporary name) is renamed to the more astute 'low-complexity jump-counting pattern'. Likewise the 'advanced jump-counting skipfield' is renamed to the 'high-complexity jump-counting pattern' - for reasoning behind this go [here](#). Both refer to time complexity of operations, as opposed to algorithmic complexity. Some other corrections.
- R9: Link to Bentley pattern paper added, and is spellchecked now.
- R8: Correction to SIMD info. Correction to structure (missing appendices title, member functions and technical specification were conjoined, acknowledgments section had mysteriously gone missing since an earlier version, now restored and updated). Update intro. HTML corrections.
- R7: Minor changes to member functions.
- R6: Re-write. `Reserve()` and `shrink_to_fit()` removed from specification.
- R5: Additional note for reserve, re-write of introduction.
- R4: Addition of revision history and review feedback appendices. General rewording. Cutting of some dead wood. Addition of some more dead wood. Reversion to HTML, benchmarks moved to external URL, based on feedback. Change of font to Times New Roman based on looking at what other papers were using, though I did briefly consider Comic Sans. Change to insert specifications.
- R3: Jonathan Wakely's extensive technical critique has been actioned on, in both documentation and the reference implementation. "Be clearer about what operations this supports, early in the paper." - done (V. Technical Specifications). "Be clear about the $O()$ time of each operation, early in the paper." - done for main operations, see V. Technical Specifications. Responses to some other feedbacks included in the foreword.
- R2: Rewording.

I. Introduction

hive

blocks with erasures list head

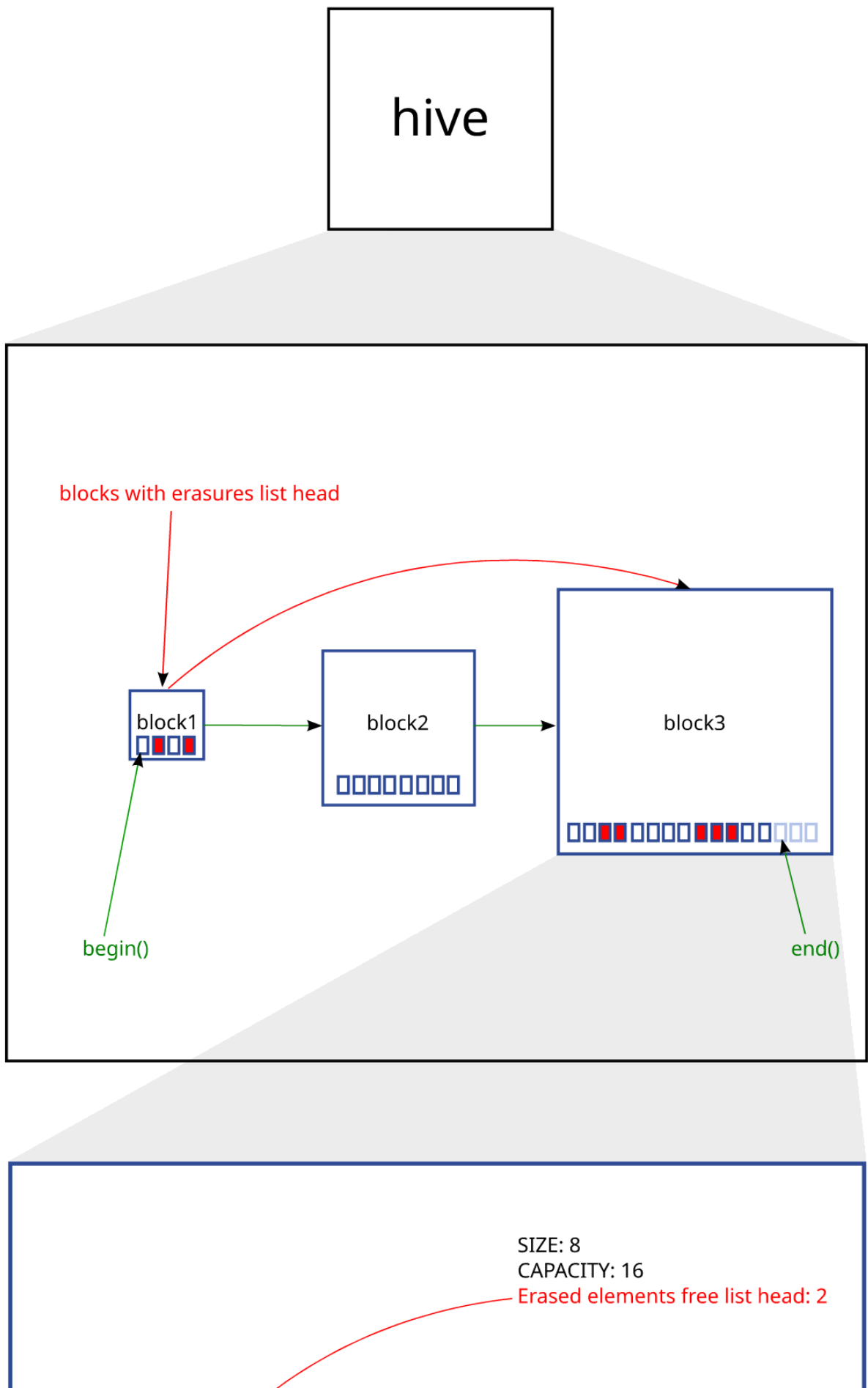
begin()

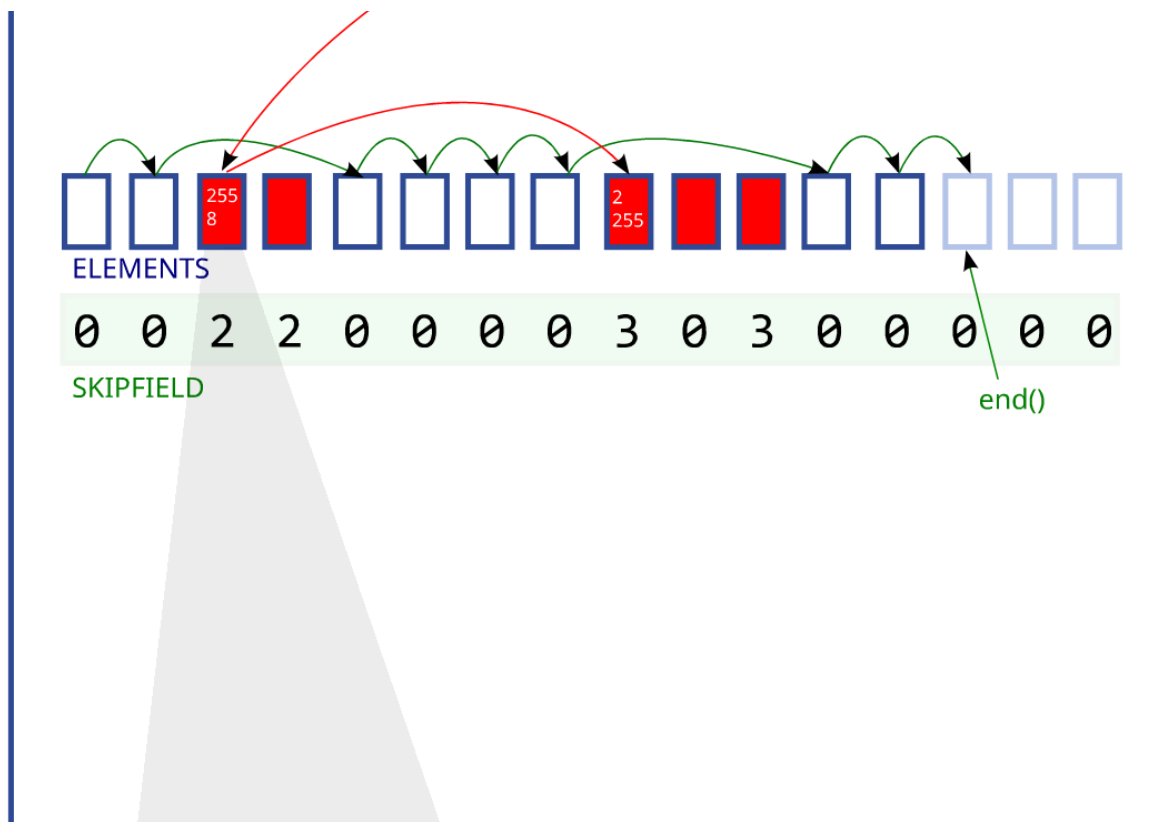
end()

SIZE: 8

CAPACITY: 16

Erased elements free list head: 2





Union/reinterpret_cast erased element memory space to record locations of previous/next nodes in the free list of runs of erased elements (for use during insertion, to re-use erased element memory locations).

A value of `numeric_limits<skipfield_type>::max()` indicates free list head is previous node in free list, when in "previous" slot. When that number is in the "next" slot, it indicates end of free list.

* NOTE: This visualises the approach taken by the reference implementation and is supplied for the purpose of giving a general understanding of structure, but other approaches are possible, see Design Decisions.

The purpose of a container in the standard library cannot be to provide the optimal solution for all scenarios. Inevitably in fields such as high-performance trading or gaming, the optimal solution within critical loops will be a custom-made one that fits that scenario perfectly. However, outside of the most critical of hot paths, there is a wide range of application for more generalized solutions.

Hive is a formalisation, extension and optimization of what is typically known as a 'bucket array' container in game programming circles. Thanks to all the people who've come forward in support of the paper over the years, I know that similar structures exist in various incarnations across many fields including high-performance computing, high performance trading, 3D simulation, physics simulation, robotics, server/client application and particle simulation fields (see:

<https://groups.google.com/a/isocpp.org/forum/#!topic/sg14/1iWHyVnsLBQ>, the [hive supporting paper #1](#) and the [appendix links to prior art](#)).

The concept of a bucket array is: you have multiple memory blocks of elements, and a boolean token for each element which denotes whether or not that element is 'active' or 'erased', commonly known as a skipfield. If it is 'erased', it is skipped over during iteration. When all elements in a block are erased, the block is removed, so that iteration does not lose performance by having to skip empty blocks. If an insertion occurs when all the blocks are full, a new memory block is allocated.

The advantages of this structure are as follows: because a skipfield is used, no reallocation of elements is necessary upon erasure. Because the structure uses multiple memory blocks, insertions to a full container also do not trigger reallocations. This means that element memory locations stay stable and iterators stay valid regardless of erasure/insertion. This is highly desirable, for example, [in game programming](#) because there are usually multiple elements in different containers which need to reference each other during gameplay and elements are being inserted or erased in real time. The only non-associative standard library container which also has this feature is `std::list`, but it is undesirable for performance and memory reasons. This does not stop it being used in [many open-source projects](#), often due to this feature.

Problematic aspects of a typical bucket array are that they tend to have a fixed memory block size, do not re-use memory locations from erased elements, and utilize a boolean skipfield. The fixed block size (as opposed to block sizes with a growth factor) and lack of erased-element re-use leads to far more allocations/deallocations than is necessary. Given that allocation is a costly operation in most operating systems, this becomes important in performance-critical environments. The boolean skipfield makes iteration time complexity undefined, as there is no way of knowing ahead of time how many erased elements occur between any two non-erased elements. This can create variable latency during iteration. It also requires branching code, which may cause issues on processors with deep pipelines and poor branch-prediction failure performance.

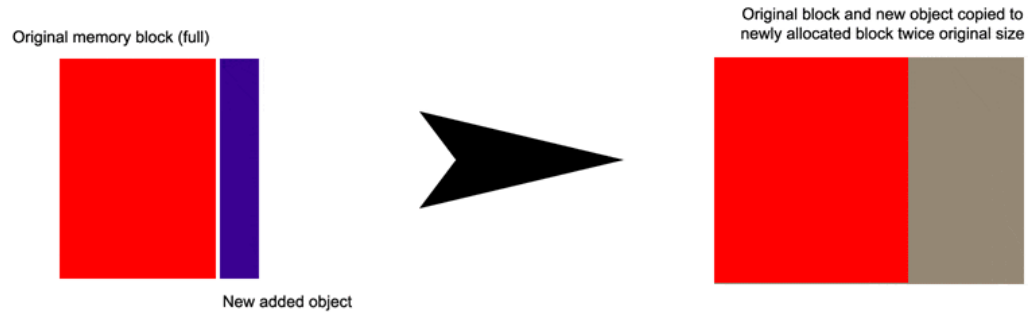
A hive uses a non-boolean method for skipping erased elements, which allows for $O(1)$ amortized iteration time complexity and more-predictable iteration performance than a bucket array. It also utilizes a growth factor for memory blocks and reuses erased element locations upon insertion, which leads to fewer allocations/reallocations. Because it reuses erased element memory space, the exact location of insertion is undefined. In most implementations it's likely (for performance reasons) that unless no erasures have occurred or an equal number of erasures and insertions have occurred, the insertion location would be the back of the container. The container is therefore considered unordered but sortable. Lastly, because there is no way of predicting in advance where erasures ('skips') may occur during iteration, an $O(1)$ time complexity `[ ]` operator is not necessarily possible (depending on implementation) and therefore, the container is bidirectional but not random-access.

There are two patterns for accessing stored elements in a hive: the first is to iterate over the container and process each element (or skip some elements using the `advance/prev/next/iterator ++/--` functions). The second is to store the iterator returned by the `insert()` function (or a pointer derived from the iterator) in some other structure and access the inserted element in that way. To better understand how insertion and erasure work in a hive, see the following images.

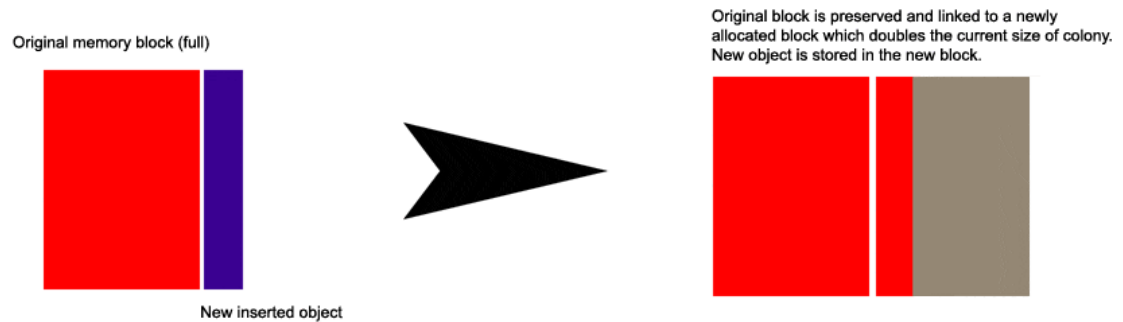
Insertion to back

The following images demonstrate how insertion works in a hive compared to a vector when size == capacity (note: images use old name for this proposal, colony. it is the same container).

Vector:



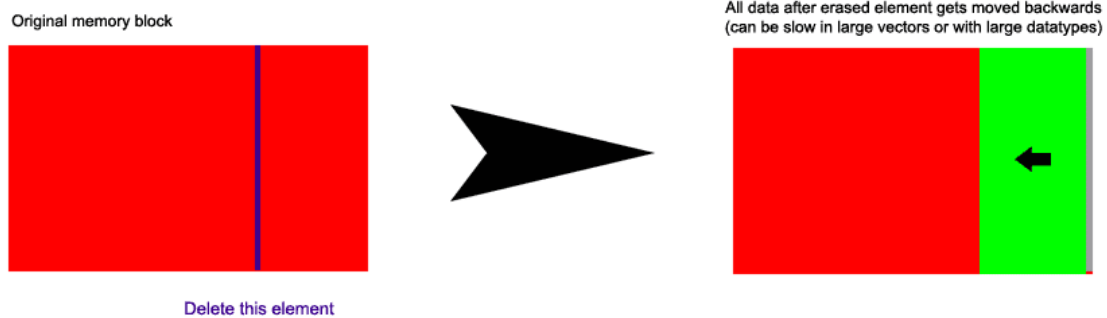
Colony:



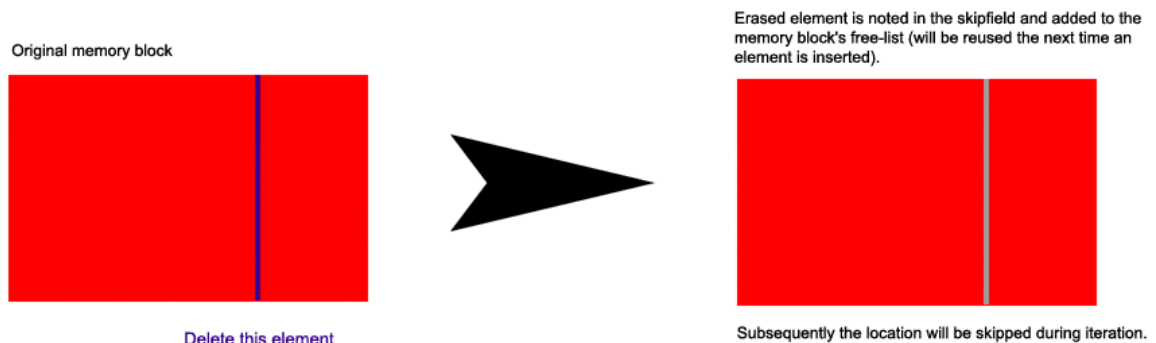
Non-back erasure

The following images demonstrate how non-back erasure works in a hive compared to a vector.

Vector:



Colony:



There is additional introductory information about the container's structure in [this CPPcon talk](#), though some of its information is out of date (hive/colony no longer uses a stack but a free list instead, benchmark data is out of date, etc), and more detailed implementation information is available in [this CPPnow talk](#).

II. Questions for the Committee

None at present.

III. Motivation and Scope

Note: Throughout this document I will use the term 'link' to denote any form of referencing between elements whether it be via ids/iterators/pointers/indexes/references/etc.

There are situations where data is heavily interlinked, iterated over frequently, and changing often. An example is the typical video game engine. Most games will have a central generic 'entity' or 'actor' class, regardless of their overall schema (an entity class does not imply an [ECS](#)). Entity/actor objects tend to be 'has a'-style objects rather than 'is a'-style objects, which link to, rather than contain, shared resources like sprites, sounds and so on. Those shared resources are usually located in separate containers/arrays so that they can re-used by multiple entities. Entities are in turn referenced by other structures within a game engine, such as quadrees/octrees, level structures, and so on.

Entities may be erased at any time (for example, a wall gets destroyed and no longer is required to be processed by the game's engine, so is erased) and new entities inserted (for example, a new enemy is spawned). While this is all happening the links between entities, resources and superstructures such as levels and quadrees, must stay valid in order for the game to run. The order of the entities and resources themselves within the containers is, in the context of a game, typically unimportant, so an unordered container is okay.

Unfortunately the container with the best iteration performance in the standard library, `vector`^[1], loses pointer validity to elements within it upon insertion, and pointer/index validity upon erasure. This tends to lead to sophisticated and often restrictive workarounds when developers attempt to utilize `vector` or similar containers under the above circumstances.

`std::list` and the like are not suitable due to their poor locality, which leads to poor cache performance during iteration. This [does not stop them](#) from being used extensively. This is however an ideal situation for a container such as `hive`, which has a high degree of locality. Even though that locality can be punctuated by gaps from erased elements, it still works out better in terms of iteration performance^[1] than every existing standard library container other than `deque`/`vector`, regardless of the ratio of erased to non-erased elements.

Some more specific requirements for containers in the context of game development are listed in the [appendix](#).

As another example, particle simulation (weather, physics etcetera) often involves large clusters of particles which interact with external objects and each other. The particles each have individual properties (spin, momentum, direction etc) and are being created and destroyed continuously. Therefore the order of the particles is unimportant, what is important is the speed of erasure and insertion. No current standard library container has both strong insertion and non-back erasure speed, so again this is a good match for `hive`.

[Reports from other fields](#) suggest that, because most developers aren't aware of containers such as this, they often end up using solutions which are sub-par for iterative performance such as `std::map` and `std::list` in order to preserve pointer validity, when most of their processing work is actually iteration-based. So, introducing this container would both create a convenient solution to these

situations, as well as increasing awareness of better-performing approaches in general. It will also ease communication across fields, as opposed to the current scenario where each field uses a similar container but each has a different name for it.

IV. Impact On the Standard

This is purely a library addition, requiring no changes to the language.

V. Design Decisions

The three core aspects of a hive from an abstract perspective are:

1. A collection of element memory blocks + metadata, to prevent reallocation during insertion (as opposed to a single memory block)
2. A method of skipping erased elements in $O(1)$ time during iteration (as opposed to reallocating subsequent elements during erasure)
3. An erased-element location recording mechanism, to enable the re-use of memory from erased elements in subsequent insertions, which in turn increases cache locality and reduces the number of block allocations/deallocations

Each memory block houses multiple elements. The metadata about each block may or may not be allocated with the blocks themselves (could be contained in a separate structure). This metadata should include at a minimum, the number of non-erased elements within each block and the block's capacity - which allows the container to know when the block is empty and needs to be removed from the iterative chain, and also allows iterators to judge when the end of one block has been reached. A non-boolean method of skipping over erased elements during iteration while maintaining $O(1)$ amortized iteration time complexity is required (amortized due to block traversal, which would typically require a few more operations). Finally, a mechanism for keeping track of elements which have been erased must be present, so that those memory locations can be reused upon subsequent element insertions.

The following aspects of a hive must be implementation-defined in order to allow for variance and possible performance improvement, and to conform with possible changes to C++ in the future:

- the method used to skip erased elements
- time complexity of operations to update whatever metadata is associated with the skip method
- erasure-recording mechanism
- element memory block metadata

- iterator structure
- memory block growth factor
- time complexity of advance()/next()/prev()

However the implementation of these *is* significantly constrained by the requirements of the container (lack of reallocation, stable pointers to non-erased elements regardless of erasures/insertions).

In terms of the [original reference implementation](#) (current reference implementation [here](#)) the specific structure and mechanisms have changed many times over the course of development, however the interface to the container and its time complexity guarantees have remained largely unchanged (with the exception of the time complexity for updating skipfield nodes - which has not impacted significantly on performance). So it is reasonably likely that regardless of specific implementation, it will be possible to maintain this general specification without obviating future improvements in implementation, so long as time complexity guarantees for the above list are implementation-defined.

Below I explain the reference implementation's approach in terms of the three core aspects described above, along with descriptions of some alternative implementation approaches.

1. Collection of element memory blocks + metadata

In the reference implementation this is essentially a doubly-linked list of 'group' structs containing (a) a dynamically-allocated element memory block, (b) memory block metadata and (c) a dynamically-allocated skipfield. The memory blocks and skipfields in this implementation have a growth factor of 2 from one group to the next. The metadata includes information necessary for an iterator to iterate over hive elements, such as the last insertion point within the memory block, and other information useful to specific functions, such as the total number of non-erased elements in the node. This approach keeps the operation of freeing empty memory blocks from the hive container at $O(1)$ time complexity. Further information is available [here](#).

Other implementations are possible. A vector of memory blocks would not work as it would disallow a growth factor in the memory blocks, but an approach using a vector-of-pointers to groups is possible (see [Appendix L](#) for a full explanation).

A short note about the exchange of user-defined block capacity limits between hives: these generally follow the pattern for allocators - which makes sense as they may have a relationship with user-supplied allocator constraints. They're transferred during move and copy construction, operator = && and swap, but not during splice or operator = &. However they're also transferred during copy construction. Unlike allocators, there is no option to specify new capacity limits in the copy/move constructor, however this makes sense for the move constructor, because specifying limits would cause the constructor to throw if the transferred blocks did not fit within the new limits. If the user wants to specify new capacity limits when copying from another hive, they do the following instead of calling a copy constructor: `hive<int> h(hive_limits(10,50)); h = other_hive;`

Likewise if they want to specify new capacity limits when moving from another hive, they can:
`hive<int> h(std::move(other_hive)); r.reshape(10, 50);`

2. A method of skipping erased elements in O(1) time during iteration

The reference implementation currently uses a skipfield pattern called the [Low complexity jump-counting pattern](#). This effectively encodes the length of runs of consecutive erased elements, into a skipfield, which allows for O(1) time complexity during iteration. Since there is no branching involved in iterating over the skipfield aside from end-of-block checks, it can be less problematic computationally than a boolean skipfield (which has to branch for every skipfield read) in terms of CPUs which don't handle branching or branch-prediction failure efficiently (eg. Core2). It also does not have the variable latency associated with a boolean skipfield.

The pattern stores and modifies the run-lengths during insertion and erasure with O(1) time complexity. It has a lot of similarities to the [High complexity jump-counting pattern](#), which was a pattern previously used by the reference implementation. Using the High complexity jump-counting pattern is an alternative, though the skipfield update time complexity guarantees for that pattern are at-worst O(n) in the number of elements in the block in question, for each insertion/erasure - however since the blocks have a fixed maximum capacity, this is technically O(1) (from the understanding which has been conveyed to me), since it does not scale with the number of elements. In practice those updates result in one memcopy operation which may resolve to a much smaller number of SIMD copies at the hardware level. But it is still slightly slower than the Low complexity jump-counting pattern (around 1-2%). The method you use to skip erased elements will typically also have an effect on the type of memory-reuse mechanism you can utilize.

A pure boolean skipfield is not usable because it makes iteration time complexity undefined - it could for example result in thousands of branching statements + skipfield reads for a single ++ operation in the case of many consecutive erased elements. In the high-performance fields for which this container was initially designed, this brings with it unacceptable latency. However another strategy using a combination of a jump-counting *and* boolean skipfield, which saves memory at the expense of computational efficiency, is possible as follows:

1. Instead of storing the data for the low complexity jump-counting pattern in its own skipfield, have a boolean bitfield indicating which elements are erased. Store the jump-counting data in the erased element's memory space instead (possibly alongside free list data).
2. When iterating, check whether the element is erased or not using the boolean bitfield; if it is not erased, do nothing. If it is erased, read the jump value from the erased element's memory space and skip forward the appropriate number of nodes both in the element memory block and the boolean bitfield.

This approach has the advantage of still performing O(1) iterations from one non-erased element to the next, unlike a pure boolean skipfield approach, but compared to a pure jump-counting approach introduces 3 additional costs per iteration via (1) a branch operation when checking the bitfield, (2) an additional read (of the erased element's memory space) and (3) a bitmasking operation + bitshift to read the bit. But it does reduce the memory overhead of the skipfield to 1 bit per-element. In the

early days of hive/colony I experimented with using both byte-based boolean skipfields and bit-based boolean skipfields. The bit-based ones were always slower, regardless of the technique. And the jump-counting skipfield was faster than both of those.

Another method worth mentioning is the use of a referencing array - for example, having a vector of elements, together with a vector of either indexes or pointers to those elements. When an element is erased, the vector of elements itself is not updated - no elements are reallocated. Meanwhile the referencing vector is updated and the index or pointer to the erased element is erased. When iteration occurs it iterates over the referencing vector, accessing each element in the element vector via the indexes/pointers. The disadvantages of this technique are (a) much higher memory usage, particularly for small elements and (b) highly-variable latency during erasure due to reallocation in the referencing array. Since one of the goals of hive is predictable latency, this is likely not suitable.

[Packed arrays](#) are not worth mentioning as the iteration method is considered separate from the referencing mechanism, making them unsuitable for a `std::` container.

3. Erased-element location recording mechanism

There are two valid approaches here; both involve per-memory-block [free lists](#), utilizing the memory space of erased elements. The first approach forms a free list of all erased elements. The second forms a free list of the first element in each *run* of consecutive erased elements ("skipblocks", in terms of the terminology used in the jump-counting pattern papers). The second can be more efficient, but requires a doubly-linked free list rather than a singly-linked free list, at least with a low-complexity jump-counting skipfield - otherwise it would become an $O(N)$ operation to update links in the skipfield, when a skipblock expands or contracts during erasure or insertion.

The reference implementation currently uses the second approach, using three things to keep track of erased element locations:

- a. Metadata for each memory block includes a 'next block with erasures' pointer and a 'previous block with erasures' pointer. The container itself contains a 'blocks with erasures' list-head pointer. These are used by the container to create an intrusive doubly-linked list of memory blocks with erased elements which can be re-used for future insertions.
- b. Metadata for each memory block also includes a 'free list head' index number, which records the index (within the memory block) of the first element of the last-created skipblock - the 'head' skipblock.
- c. The memory space of the first erased element in each skipblock is reinterpret_cast'd via pointers as two index numbers, the first giving the index of the previous skipblock in that memory block, the second giving the index of the next skipblock in the sequence. In the case of the 'head' skipblock in the sequence, a unique number is used for the 'next' index. This forms a free list of runs of erased element memory locations which may be re-used.

Using indexes for next and previous links, instead of pointers, reduces the necessary bit-depth of the next and previous links, thereby reducing the necessary over-alignment of the container's element

type in order to support the free list links. If a global (ie. not per-memory-block) free list of erased elements or skipblocks was used, pointers would be necessary instead of indexes, as hive is bidirectional and does not support the `[ ]` operator. This would increase the minimum alignment necessary for the hive element type to $\text{sizeof}(\text{pointer}) * 2$. A global free list would also decrease cache locality when traversing the free list by jumping between memory blocks. And in addition, when a block was removed from the iterative sequence (sequence of blocks which are interacted with when iterating across the container) upon becoming empty it would force an $O(n)$ traversal of the free-list to find all erased elements (or skipblocks) within that particular block and remove them from the free list. This is obviously unacceptable for performance and latency reasons.

Previous versions of the reference implementation used a singly-linked free list of erased elements instead of a doubly-linked free list of skipblocks. This was possible with the High complexity jump-counting pattern, but not possible using the Low complexity jump-counting pattern as it cannot calculate a skipblock's start node location from a middle node's value like the High complexity pattern can. But using free-lists of skipblocks is a more efficient approach as it requires fewer free list nodes. In addition, re-using only the start or end nodes of a skipblock is faster because it never splits a single skipblock in two (which would require adding a new skipblock to the free list).

An example of why a doubly-linked free list is necessary, is when you erase an element which is in between two skipblocks. In that case two skipblocks must be combined into one skipblock, and the previous secondary skipblock must be removed from that block's free list of skipblocks. If the free list is singly-linked, the hive must do a linear search through the free list to find the skipblock prior to the secondary skipblock mentioned, in order to update that free list node's "next" index link. This is at worst $O(n)$ in the number of skipblocks within that block. However if a doubly-linked free list is used, the prior skipblock is linked to from the secondary skipblock mentioned, making updating all free list links $O(1)$. One could however revert to a singly-linked free list of skipblocks for very small `value_type`'s, using a high-complexity jump-counting skipfield, in order to reduce the overalignment necessary to store both free list nodes in the element memory space.

One cannot use a stack of pointers (or similar) to erased elements for this mechanism, as early versions of the reference implementation did, because this can create allocations during erasure, which changes the exception guarantees of `erase()`. One could instead scan all skipfields until an erased location was found, or simply have the first item in the list above and then scan the first available block, though both of these approaches would be slow.

In terms of the alternative *boolean + jump-counting skipfield* approach described in the erased-element-skip-method section above, one could store both the jump-counting data and free list data in any given erased element's memory space, provided of course that elements are aligned to be wide enough to fit both.

Note here that I have mainly been talking about how to keep track of erasures within blocks, not which blocks have erasures. In the reference implementation this is achieved by having an intrusive linked list of the groups whose blocks contained erasures, as mentioned. This increases group metadata memory usage by two pointers. Other potential methods include:

- a. Storing a vector of pointers to groups whose blocks contain erasures. To preserve `erase()` exception guarantees, this vector would have to be expanded upon insertion, not erasure, and thus it's capacity should always be $\geq$ the number of blocks. When a group becomes completely

empty of elements and is removed from the iterative chain, its entry in the vector is swapped to the back and popped. This approach reduces the additional memory cost down to 1 pointer per group.

- b. Storing a record of the last one-or-more groups whose blocks have had erasures and then re-using from those groups during insertion, until they are full and then (on subsequent insertions) searching consecutive groups until a group with erasures is found. The records would be updated every time an erasure is made. The assumption is made that groups close to the recorded group(s) with erasures are more likely to have erasures, but this depends on the erasure pattern. An alternative strategy would be to search backward from the last group in the hive, as later groups are likely to be larger and therefore statistically may contain more erasures. This is the approach used by `plf::list`, but it is efficient there because the main structure is a vector of groups, hence the search phase is cache-friendly, even though the worst case scenario is $O(n)$ in the number of groups. In `hive` it is not cache-friendly because each group must be allocated individually (see [Appendix L](#) for why), so while this approach reduces the cost of keeping track of which groups have erasures down to 2 pointers per hive, it will reduce performance and increase latency variability during insertion due to the search phase.
- c. If a vector-of-pointers-to-groups approach is taken (see [Appendix L](#)) one can either 1. construct a jump-counting skipfield, which skips the groups which Do Not contain erasures, and use this to find the first group with erasures in $O(1)$ time, or 2. construct a bitfield indicating which groups have erasures and linearly search through it. These approaches reduce the cost down to either 1 `size_type` per group or 1 bit per group respectively.

A final note: due to the strong variability in terms of time complexity requirements between the different ways of implementing these three core aspects of the container, where there likely to be significant variance based on implementation the time complexity of each implementation will not be included in the technical specification.

Implementation of iterator class

Any iterator implementation is going to be dependent on the erased-element-skipping mechanism used. The reference implementation's iterator stores a pointer to the current 'group' struct mentioned above, plus a pointer to the current element and a pointer to its corresponding skipfield node. An alternative approach is to store the group pointer + an index, since the index can indicate both the offset from the memory block for the element, as well as the offset from the start of the skipfield for the skipfield node. However multiple implementations and benchmarks across many processors have shown this to be worse-performing than the separate pointer-based approach, despite the increased memory cost for the iterator class itself.

++ operation is as follows, utilising the reference implementation's Low-complexity jump-counting pattern:

1. Add 1 to the existing element and skipfield pointers.
2. Dereference skipfield pointer to get value of skipfield node, add value of skipfield node to both the skipfield pointer and the element pointer. If the node is erased, its value will be a positive

integer indicating the number of nodes until the next non-erased node, if not erased it will be zero.

3. If element pointer is now beyond end of element memory block, change group pointer to next group, element pointer to the start of the next group's element memory block, skipfield pointer to the start of the next group's skipfield. In case there is a skipblock at the beginning of this memory block, dereference skipfield pointer to get value of skipfield node and add value of skipfield node to both the skipfield pointer and the element pointer. There is no need to repeat the check for end of block, as the block would have been removed from the iteration sequence if it were empty of elements.

-- operation is the same except both step 1 and 2 involve subtraction rather than adding, and step 3 checks to see if the element pointer is now before the beginning of the memory block. If so it traverses to the back of the previous group, and subtracts the value of the back skipfield node from the element pointer and skipfield pointer.

Iterators are bidirectional but also provide constant time complexity $>$, $<$, $>=$, $<=$ and $<=>$ operators for convenience (eg. in for loops when skipping over multiple elements per loop and there is a possibility of going past a pre-determined end element). This is achieved by keeping a record of the order of memory blocks. In the reference implementation this is done by assigning a number to each memory block in its metadata. In an implementation using a vector of pointers to memory blocks instead of a linked list, one could use the position of the pointers within the vector to determine this. Comparing relative order of the two iterators' memory blocks via this number, then comparing the memory locations of the elements themselves, if they happen to be in the same memory block, is enough to implement all greater/lesser comparisons.

Additional notes on specific functions

- Non-member function specializations for **advance**, **prev** and **next** (all variants)

For these functions, complexity is dependent on state of hive, position of iterator and amount of distance, but in many cases will be less than linear, and may be constant. To explain: it is necessary in a hive to store metadata both about the capacity of each block (for the purpose of iteration) and how many non-erased elements are present within the block (for the purpose of removing blocks from the iterative chain once they become empty). For this reason, intermediary blocks between the iterator's initial block and its final destination block (if these are not the same block, and if the initial block and final block are not immediately adjacent) can be skipped rather than iterated linearly across, by using the "number of non-erased elements" metadata.

This means that the only linear time operations are any iterations within the initial block and the final block. However if either the initial or final block have no erased elements (as determined by comparing whether the block's capacity metadata and the block's "number of non-erased elements" metadata are equal), linear iteration can be skipped for that block and pointer/index math used instead to determine distances, reducing complexity to constant time. Hence the best case for this operation is constant time, the worst is linear to the distance.

A special case for the latter operation is the back block, where instead of comparing number of non-erased elements with capacity, we compare it with the distance between the start of the block and `end()`. Depending on implementation there may be other ways to check whether the block is empty (in the reference implementation we can check to see whether the head of the block's free list of erased element locations has the magic value of `numeric_limits<skipfield_type>::max()`). But the method above will be available to any implementation for the reasons described.

- **Non-member function specialization for `distance`** (all variants)

The same considerations which apply to `advance`, `prev` and `next` also apply to `distance` - intermediary blocks between first and last's blocks can be skipped in constant time and their "number of non-erased elements" metadata added to the cumulative distance count, while first's block and last's block (if they are not the same block) must be linearly iterated across unless either block has no erased elements, in which case the operation becomes pointer/index math and is reduced to constant time for that block. In addition, if first's block is not the same as last's block, and last is equal to `end()` or `--end()`, or is the last element in it's block, last's block's elements can also counted from the "number of non-erased elements" metadata rather than via iteration. If first and last are in the same block but are in the first and last element slots in the block, distance can again be calculated from the "number of non-erased elements" metadata for that block.

- **`iterator insert`** (all variants)

Insertion re-uses previously-erased element memory locations when available, so position of insertion is effectively random unless no previous erasures have occurred, in which case all elements will be inserted linearly to the back of the container, at least in the current implementation. These details have been removed from the standard in order to allow leeway for potentially-better implementations in future - though it is expected that a hive will always reuse erased memory locations, it is impossible to predict optimal strategies for unknown future hardware.

As the technical specification says, this operation may invalidate iterators pointing to the past-the-end iterator. It should be noted that this only occurs when inserting to the back of the container so when an implementation re-uses erased element memory spaces, the location of `end()` does not change. If the previous location of `end()` was beyond the end of a memory block, then iterators which were previously pointing to it may not be usable post-insertion.

The same cannot be said about `begin()`, the location of which may also change when `insert()` is called (if elements were erased before the current `begin()` location, and those memory spaces are reused). `begin()` always pointed to an active element, and an iterator which pointed to `begin()` prior to `insert()` may no longer be pointing to `begin()`, but is still pointing to an active element so is still usable.

- **`void insert`** (all variants)

For `range`, `fill` and `initializer_list` insertion, it is not possible to guarantee that all the elements inserted will be sequential in the hive's iterative sequence of blocks, and therefore it is not considered useful to return an iterator to the first inserted element. There is a precedent for this in the various `std::map` containers. Therefore these functions return `void` presently.

In the case of an ordered insertion container, returning an iterator makes sense as the insert function is returning an iterator to the start of the inserted range, and the user can then iterate over that range. In hive, the range that is inserted can be distributed across multiple areas in the container, so when you return an iterator, all you do is give the user an iterator to the first item from the inserted sequence, not an iterator to the start of an iterable sequence which matches the input sequence. As such, returning an iterator *Will* be confusing to users and *Should* be disabled.

The same considerations regards iterator invalidation in singular insertion above, also apply to these inserts.

- **iterator erase(const_iterator position);**

Firstly it should be noted that erase may retain memory blocks which become completely empty of elements due to erasures, adding them to the set of unused memory blocks which are normally created by reserve(). Under what circumstances these memory blocks are retained rather than deallocated is implementation-defined - however given that small memory blocks have low cache locality compared to larger ones, from a performance perspective it is best to only retain the larger of the blocks currently allocated in the hive. In most cases this would mean the back block would almost always be retained. There is a lot of nuance to this, and it's also a matter of trading off complexity of implementation vs actual benchmarked speed vs latency. In my tests retaining both back blocks and 2nd-to-back blocks while ignoring actual capacity of blocks seems to have the best overall performance characteristics.

There are three major performance advantages to retaining back blocks as opposed to any block - the first is that these will be, under most circumstances, the largest blocks in the hive (given the built-in growth factor) - the only exception to this is when splice is used, which may result in a smaller block following a larger block (implementation-dependent). Larger blocks == more cache locality during iteration, large numbers of erased elements notwithstanding. The second advantage is that in situations where elements are being inserted to and erased from the back of the hive (this assumes no erased element locations in other memory blocks, which would otherwise be used for insertions) continuously and in quick succession, retaining the back block avoids large numbers of deallocations/reallocations. The third advantage is that deallocations of larger blocks can, in part, be moved to non-critical code regions via trim_capacity(). Though ultimately if the user wants total control of when allocations and deallocations occur they would want to use a custom allocator.

A final note about time complexity. In the technical specification there is a note stating that the time complexity of updating the erased-element-skipping mechanism (probably a skipfield of some description) is not factored into the stated time complexity and that it can be constant, linear or otherwise defined. This is to allow of leniency in terms of implementation, but also because abstract time complexity does not always describe hardware time complexity. When the reference implementation was using the high-complexity jump-counting skipfield, it's time complexity for updating the skipfield on erase was linear in the number of skipfield nodes affected (which is impossible to predict from the users's perspective, making this effectively undefined) - however these updates were almost entirely executed in terms of singular memcpy operations which on a hardware level are often a much smaller number of copies using AVX or similar.

Because of this, changing to the low-complexity jump-counting skipfield pattern (which always has abstract $O(1)$ updates) did not affect latency and performance as much as might be expected, though it did affect performance. Hence we need to allow leniency in this area and trust developers to do the right thing in case they come up with a faster erased-element skipping mechanism which doesn't necessarily have $O(1)$ time from an abstract point of view.

- **`iterator erase(const_iterator first, const_iterator last)`**

The same considerations for singular erasure above also apply for range-erasure. Algorithmically, ranged erasure is $O(N)$ if elements are non-trivially-destructible. If they are trivially-destructible, we follow similar logic to the distance specialization above. That is to say, for the first and last blocks in the range, if the number of erased elements in that block is equal to capacity, there are no erasures in the block and we can simply create a skipblock in the skipfield without checking the skipfield values. If there are erasures in that block, we need to process the part of the skipfield which is within the range to identify which elements within the range are already erased, so that we can update `size()` correctly.

For any intermediary blocks between the first and last blocks, we can simply deallocate or reserve them without calling destructors of elements or processing skipfields. Hence, for trivially-destructible types, the entire operation can be linear in the number of blocks contained within the range (best case $O(1)$), or linear in the number of elements contained within the range ($O(N)$).

Lastly, specifying a return iterator for range-erase may seem pointless, as no reallocation of elements occurs in erase so the return iterator will almost always be the `last` iterator of the `const_iterator first, const_iterator last` pair. However if `last` was `end()`, the new value of `end()` (if it has changed due to empty block removal) will be returned. In this case either the user submitted `end()` as `last`, or they incremented an iterator pointing to the final element in the hive and submitted that as `last`. The latter is the only valid reason to return an iterator from the function, as it may occur as part of a loop which is erasing elements which ends when `end()` is reached. If `end()` is changed by the erasure of an entire memory block, but the iterator being used in the loop does not accurately reflect `end()`'s new value, that iterator could iterate past `end()` and the loop would never finish.

- **`void reshape(std::hive_limits block_limits);`**

This function updates the block capacity limits in the hive and, if necessary, changes any blocks which fall outside of those limits to be within the limits. For this reason it may trigger an exception with non-copyable/movable types, and also invalidate pointers/iterators/etc to elements.

The order of elements post-reshape is not guaranteed to be stable in order to allow for optimizations. Specifically in the instance where a given element memory block no longer fits within the limits supplied by the user, depending on the state of the hive as a whole, the elements within that memory block could be reallocated to previously-erased element locations in other memory blocks which do fit within the supplied limits. Or they could be reallocated to the back of the final memory block.

Additionally if there is empty capacity at the back of the last block in the container, at least some of the elements could be moved to that position rather than being reallocated to a new memory

block. Both of these techniques increase cache locality by removing skipped memory spaces within existing memory blocks. However whether they are used is implementation-dependent.

- **static constexpr std::hive_limits block_capacity_hard_limits() noexcept;**

As opposed to `block_capacity_limits()`, which returns the current min/max element block capacities for a given instance of `hive`, this allows the user to get any implementation's min/max 'hard' lower/upper limits for element memory block capacities ie. the limits which any user-supplied limits must fit within. For example, if an implementation's hard limit is 3 elements min, 1 million elements max, all user-supplied limits must be ≥ 3 and ≤ 1 million.

This is useful for 2 reasons:

- a. An implementation may have default block capacity limits which are different from its hard limits.
- b. A user should have a mechanism for determining what user-defined limits they can supply without triggering an exception, at run-time/compile-time, so that a project can run on multiple standard library implementations.

- **void clear();**

User expectation was that `clear()` would erase all elements but not deallocate memory blocks. If deallocation of memory blocks was desired, a `clear()` call can be followed by a `trim_capacity()` call. The time complexity for this in terms of the reference implementation is constant for trivially-destructible types and linear in the size of the sequence for non-trivially-destructible types. However in an implementation using a vector of pointers to blocks+metadata rather than a linked list, the time taken to transfer active blocks to the reserved block vector would be linear in the number of active blocks in the sequence. Hence the tech spec allows some leeway here and does not specify as constant for trivially-destructible types.

- **iterator get_iterator(const_pointer p) noexcept;**
const_iterator get_iterator(const_pointer p) const noexcept;

Because `hive` iterators are likely to be large, storing three pieces of data - current memory block, current element within memory block and potentially, current skipfield node - a program storing many links to elements within a `hive` may opt to dereference iterators to get pointers and store those instead of iterators, to save memory. This function reverses the process, giving an iterator which can then be used for operations such as `erase`. `get_const_iterator` was fielded as a workaround for the possibility of someone wanting to supply a non-const pointer and get a `const_iterator` back, however `as_const` fulfills this same role when supplied to `get_iterator` and doesn't require expanding the interface of `hive`. Likewise if a user wishes to supply a pointer to `iterator get_iterator(const_pointer p)`, they can use `as_const` whereas the reverse isn't true.

Note that this function is only guaranteed to return an iterator that corresponds to the pointer supplied - it makes no checks to see whether the element which `p` originally pointed to is the same element which `p` now points to. Resolving this problem is down to the end user and could involve having a unique id within elements or something similar (more info in FAQ).

As of P0447R20 the semantics of this function have been re-written to match the standard's policy regarding the reading of pointers to erased elements. That is to say, if this function is called with a pointer to an erased element, behavior is undefined, because reading the value of a pointer to a destructed element (even without dereferencing it) could potentially crash in a given scenario on a given platform. However this function must be specified to return `end()` if `p` does not point to an element in `*this`, because of the following situation:

```
hive<int> a = {1, 2, 3};
hive<int> b = {4, 5, 6};
hive<int>::iterator c = a.begin();
int *d = &*c;
hive<int>::iterator e = b.get_iterator(d);
```

The `get_iterator` line is a perfectly valid call - except that `d` points to an element in `a`, not `b`. Hence, regardless of the erased/non-erased status of a given element, `get_iterator` must return `end()` when `d` does not match an element in `*this`. So in most situations, the function should still work as originally intended - returning `end()` if an element is erased - but this is platform-dependent and undefined behavior. I have spoken with game developers and this situation is tenable - they deal with UB all the time and it is unproblematic. However if at some point in the future it is found to be untenable, an overload can be added to take a secondary hive-internal pointer type which stores a `void*` internally and dereferences to the element type on operators `*` and `->`. P2414 is another paper dealing with this issue and problem-resolutions from that paper may factor in.

Please note that *if* an implementation does check whether a given element is erased, when using the LCJC skipfield pattern as the erased element skipping mechanism the criteria under "Parallel processing" in the [LCJC paper](#) must be met.

The function compares pointers against the start and ends of the various memory blocks in `*this`. There was some confusion that this would be problematic due to obscure rules of the standard which state that a given platform may allow addresses outside of a given memory block to have addresses that are within that memory block, at least in terms of the `std::less/std::greater` operators et al. According to Jens Maurer, these difficulties can be bypassed via hidden channels between the library implementation and the compiler.

▪ **`bool is_active(const_iterator it) const noexcept;`**

This function checks whether `it` points to a non-erased element in `*this`. First we must check that the memory block the iterator points to is part of the hive's current blocks - otherwise checking the skipfield might trigger an out-of-bounds exception, as the block might've been deallocated. Then we can directly use the iterator's skipfield pointer (or any other erased-element skipping mechanism) to check the erased/non-erased status of the element in question.

The reference implementation uses pointers to overaligned elements in iterators, and `reinterpret_cast`'s to the relevant pointer type upon dereferencing, which gets around the C++ standard's concern with reading the value of pointers to erased elements (the pointer comparisons used in this function are between the pointers to overaligned elements, which, while they point to the same memory space, critically do not point to the same *type* as the actual element which has been constructed/destructed there). I would expect any working implementation to do the same

thing (support overaligned types and/or use a different pointer type in order to accomodate the free list mechanisms).

The same criteria for checking for erased elements in `get_iterator` also applies here.

- **`void shrink_to_fit();`**

A decision had to be made as to whether this function should, in the context of `hive`, be allowed to reallocate elements (as `std::vector` does) or simply trim off unused memory blocks (as `std::deque` does). Due to the fact that a large `hive` memory block could have as few as one remaining element after a series of erasures, it makes little sense to only trim unused blocks, and instead a `shrink_to_fit` is expected to reallocate all non-erased elements to as few memory blocks as possible in order to increase cache locality during iteration and reduce memory use. As with `reshape()`, the order of elements post-`reshape` is not guaranteed to be stable, to allow for potential optimizations.

The reference implementation takes (at time of writing) a fairly brute-force approach to this function - creates a new temporary `hive` of the desired size, utilizing as few element blocks as possible, copies/moves all elements from the original `hive` into it, then swaps the `hives` and destroys the temporary. A more astute implementation might for example allocate a temporary array detailing the full capacity and unused capacity of each block, then use an algorithm to move all the elements into as few of the existing blocks as possible, filling up any erased element locations and/or unused space at the back of the `hive` - only allocating new blocks when necessary. Working out an optimal strategy in this scenario gets complicated quickly.

- **`void trim_capacity();`**
`void trim_capacity(size_type n);`

The `trim_capacity()` command was also introduced as a way to free unused memory blocks which have been previously reserved or transferred from active use to inactive use eg. via `erase()`, without reallocating elements and invalidating iterators. The second overload was introduced as a way of allowing the user to say "I want to retain at least `n` capacity while freeing unused blocks, so that I have room for future insertions without having to allocate again". The specific semantics of the function mean the user doesn't have to know how much unused capacity is actually in (a) unused element memory space at the back of the back block, (b) unused element memory space from prior erasures, or (c) in unused blocks. They just say how much they want to retain, and the implementation will free the remainder (`capacity() - n`) via deallocating unused blocks if (a) unused blocks exist and (b) they are smaller than the remainder. It is not required to reduce `capacity()` if this is not the case.

- **`void sort();`**

It is foreseen that although the container has unordered insertion, there may be circumstances where sorting is desired. Because `hive` uses bidirectional iterators, using `std::sort` or similar is not possible. Therefore an internal sort routine is warranted, as it is with `std::list`. An implementation of the sort routine used in the reference implementation of `hive` can be found in a non-container-specific form at pflib.org/indiesort.htm - see that page for the technique's advantages over the usual sort algorithms for non-random-access containers. To date there has been no interest in including this algorithm in the standard library. An allowance is made for `sort` to allocate memory if necessary, so that algorithms such as `indiesort` can be used internally.

- **void unique();**
template <class BinaryPredicate>
size_type unique(BinaryPredicate binary_pred);

Likewise, if a container can be sorted, unique may be of use post-sort. In this case optimal implementation of unique involves calling the range-erase function where possible, as range-erase has potentially constant time depending on the scenario, as opposed to calling single-element erase repeatedly.

- **void splice(hive &x);**
void splice(hive &&x);

Whether x's active blocks are transferred to the beginning or end of *this's sequence of active blocks, or interlaced in some way (for example, to preserve relative capacity growth-factor ordering of subsequent blocks) is implementation-defined. Better performance may be gained in some cases by allowing the source's active blocks to go to the front rather than the back, depending on how full the final active block in x is. This is because unused elements that are not at the back of hive's iterative sequence will need to be marked as skipped, and skipping over large numbers of elements will incur a small performance disadvantage during iteration compared to skipping over a small number of elements, due to memory locality.

This function is not noexcept for three reasons - the first is that a length_error exception may be thrown if any of the capacities of the source x's blocks are outside of the range defined by the destination's (*this) minimum and maximum block capacity limits. Second is that an exception may be thrown if the allocators of the two hives are different. Third is that in the case of an implementation using a linked list of group structs (ala the reference implementation) transferring blocks involves no allocation, however in the case of an implementation using a vector of pointers to blocks, an additional allocation may have to be made if the group pointer vector isn't of sufficient capacity to accomodate pointers to the spliced blocks from the source.

Time complexity is a complex issue here - again, for a vector-of-pointers-to-blocks implementation, we need to allow for potentially needing to expand the capacity of the vector and copying both destination and source pointers into it, hence the time complexity is linear in the number of source and destination blocks. In terms of the reference implementation the time complexity is at best constant, at worst linear in the number of source and destination blocks.

Lastly, reserved (unused) memory blocks from the source are not transferred into the destination. They are retained in the source.

- Lack of **resize()** (all variants)

This is a conscious choice to avoid confusing the user, as insertion location into a hive is undefined. In the case of hive, resizing would not necessarily insert new elements to the back of the container, when the supplied size was larger than the existing size(). New elements could be inserted into erased elements memory locations. This also means the initialization of those non-contiguous elements (if they are POD types) cannot be optimized by use of memset. This removes the main performance reason to allow for resize(). The lack of ability to specify the location of insertion removes the "ease of developer use" reason to include resize().

Results of implementation

In practical application the reference implementation is generally faster for insertion and (non-back) erasure than current standard library containers, and generally faster for iteration than any container except vector and deque. For full details, see [benchmarks](#).

VI. Technical Specification

Suggested location of hive in the standard is Sequence Containers.

Header <version> synopsis [version.syn]

```
#define __cpp_lib_hive <editor supplied value> // also in <hive>
```

General [containers.general]

Containers library summary [tab:containers.summary]

Subclause	Header
Requirements	
Sequence containers	<array>, <deque>, <forward_list>, <list>, <vector>, <hive>
Associative containers	<map>, <set>
Unordered associative containers	<unordered_map>, <unordered_set>
Container adaptors	<queue>, <stack>
Views	

Sequence containers [sequence.reqmts]

1. A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides four basic kinds of sequence containers: vector, forward_list, list, and deque. In addition, array **is provided as a sequence container which provides limited sequence operations because it has a fixed number of elements** **and hive are provided as**

sequence containers which provide limited sequence operations, in array's case because it has a fixed number of elements, and in hive's case because insertion order is unspecified. The library also provides container adaptors that make it easy to construct abstract data types, such as stacks or queues, out of the basic sequence container kinds (or out of other kinds of sequence containers that the user defines).

Header <hive> synopsis [hive.syn]

```
#include <initializer_list> // see [initializer.list.syn]
#include <compare> // see [compare.syn]

namespace std {
    // class template hive

    struct hive_limits;

    template <class T, class Allocator = allocator<T>> class
hive;

    template<class T, class Allocator>
        void swap(hive<T, Allocator>& x, hive<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template<class T, class Allocator, class U>
        typename hive<T, Allocator>::size_type
            erase(hive<T, Allocator>& c, const U& value);

    template<class T, class Allocator, class Predicate>
        typename hive<T, Allocator>::size_type
            erase_if(hive<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template <class T>
            using hive = std::hive<T, polymorphic_allocator<T>>;
    }
}
```

22.3.14 Class template hive [hive]

22.3.14.1 Class template hive overview [hive.overview]

1. A hive is a sequence container that allows constant-time insert and erase operations. Insertion position is not specified, but will in most implementations typically be the back of the container when no erasures have occurred, or when erasures have occurred it can re-use existing erased element memory locations. Storage management is handled automatically and is specifically organized in multiple blocks of elements, referred to hereon as *element blocks*.
2. Element blocks which are interacted with when iterating over the sequence are referred to as *active blocks*, those which are not are referred to as *reserved blocks*. Active blocks which become empty of sequence elements [Example: via use of `erase` or `clear` - end example] are either deallocated or become reserved blocks. Reserved blocks can be allocated by the user [Example: by `reserve` - end example] for future use during insertions, at which point they become active blocks.
3. Erasures use unspecified techniques to mark erased elements as being erased, as opposed to relocating subsequent elements during erasure as is expected in a vector or deque. These elements are subsequently skipped during iteration. The same, or different unspecified techniques may be used to record the locations of erased elements, such that those locations may be reused later during insertions. All of these techniques have constant time complexity.
4. Active block capacities have an implementation-defined growth factor (which need not be integral), for example a new active block's capacity could be equal to the summed capacities of the pre-existing active blocks.
5. Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.
 - (5.1) — The minimum limit shall be no larger than the maximum limit.
 - (5.2) — When limits are not specified by a user during construction, the implementation's default limits are used.
 - (5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks [Note 1: To allow latitude for both implementation-specific and user-directed optimization. - end note]. The latter are defined as *hard limits*.
 - (5.4) — If user-specified limits are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, behavior is undefined.
 - (5.5) — After construction a hive's limits can be changed by the user via `reshape`.
 - (5.6) — Limits are copied between hives during copy construction, move construction, `operator =` (`hive &&`) and `swap`.
 - (5.7) — An element block is said to be *within the bounds of* a pair of minimum/maximum limits when it's capacity is $\geq$ the minimum limit and $\leq$ the maximum limit.
6. A hive conforms to the requirements for Containers ([`container.reqmts`]), with the exception of operators `==` and `!=`. A hive also meets the requirements of a reversible container ([`container.rev.reqmts`]), of an allocator-aware container ([`container.alloc.reqmts`]), and some of

the requirements of a sequence container, including several of the optional sequence container requirements ([sequence.reqmts]). Descriptions are provided here only for operations on hive that are not described in that table or for operations where there is additional semantic information.

7. Hive iterators meet the Cpp17BidirectionalIterator requirements but also provide relational operators `<`, `<=`, `>`, `>=` and `<=>` which compare the relative ordering of two iterators in the sequence.

```
namespace std {

struct hive_limits
{
    size_t min;
    size_t max;
    constexpr hive_limits(size_t minimum, size_t maximum)
noexcept : min(minimum), max(maximum) {}
};

template <class T, class Allocator = allocator<T>>
class hive {
private:
    hive_limits current_limits = implementation-defined; //
    exposition only

public:

    // types
    using value_type = T;
    using allocator_type = Allocator;
    using pointer = typename
allocator_traits<Allocator>::pointer;
    using const_pointer = typename
allocator_traits<Allocator>::const_pointer;
    using reference = value_type&;
    using const_reference = const value_type&;
    using size_type = implementation-defined; // see
[container.requirements]
    using difference_type = implementation-defined; // see
[container.requirements]
    using iterator = implementation-defined; // see
[container.requirements]
    using const_iterator = implementation-defined; // see
[container.requirements]
    using reverse_iterator = std::reverse_iterator<iterator>; //
    see [container.requirements]
    using const_reverse_iterator =
```

```
std::reverse_iterator<const_iterator>; // see  
[container.requirements]
```

```
constexpr hive() noexcept(noexcept(Allocator())) :  
hive(Allocator()) { }  
explicit hive(const Allocator&) noexcept;  
explicit hive(hive_limits block_limits) : hive(block_limits,  
Allocator()) { }  
hive(hive_limits block_limits, const Allocator&);  
explicit hive(size_type n, const Allocator& = Allocator());  
hive(size_type n, hive_limits block_limits, const Allocator&  
= Allocator());  
hive(size_type n, const T& value, const Allocator& =  
Allocator());  
hive(size_type n, const T& value, hive_limits block_limits,  
const Allocator& = Allocator());  
template<class InputIterator>  
hive(InputIterator first, InputIterator last, const  
Allocator& = Allocator());  
template<class InputIterator>  
hive(InputIterator first, InputIterator last, hive_limits  
block_limits, const Allocator& = Allocator());  
template<container-compatible-range<T> R>  
hive(from_range_t, R&& rg, const Allocator& = Allocator());  
template<container-compatible-range<T> R>  
hive(from_range_t, R&& rg, hive_limits block_limits, const  
Allocator& = Allocator());  
hive(const hive& x);  
hive(hive&&) noexcept;  
hive(const hive&, const type_identity_t<Allocator>&);  
hive(hive&&, const type_identity_t<Allocator>&);  
hive(initializer_list<T> il, const Allocator& = Allocator());  
hive(initializer_list<T> il, hive_limits block_limits, const  
Allocator& = Allocator());  
  
~hive();  
hive& operator=(const hive& x);  
hive& operator=(hive&& x)  
noexcept(allocator_traits<Allocator>::propagate_on_container_mo  
ve_assignment::value ||  
allocator_traits<Allocator>::is_always_equal::value);  
hive& operator=(initializer_list<T>);  
template<class InputIterator>  
void assign(InputIterator first, InputIterator last);  
template<container-compatible-range <T> R>  
void assign_range(R&& rg);  
void assign(size_type n, const T& t);
```

```

void assign(initializer_list<T>);
allocator_type get_allocator() const noexcept;

// iterators
iterator          begin() noexcept;
const_iterator    begin() const noexcept;
iterator          end() noexcept;
const_iterator    end() const noexcept;
reverse_iterator  rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator  rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator    cbegin() const noexcept;
const_iterator    cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// capacity
[[nodiscard]] bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
size_type capacity() const noexcept;
void reserve(size_type n);
void shrink_to_fit();
void trim_capacity() noexcept;
void trim_capacity(size_type n) noexcept;

// modifiers
template <class... Args> iterator emplace(Args&&... args);
iterator insert(const T& x);
iterator insert(T&& x);
void insert(size_type n, const T& x);
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<container-compatible-range <T> R>
    void insert_range(R&& rg);
void insert(initializer_list<T> il);

iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void swap(hive&)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);

```

```

void clear() noexcept;

// hive operations
void splice(hive& x);
void splice(hive&& x);
size_type unique();
template<class BinaryPredicate>
    size_type unique(BinaryPredicate binary_pred);

hive_limits block_capacity_limits() const noexcept;
static constexpr hive_limits block_capacity_hard_limits()
noexcept;
void reshape(hive_limits block_limits);

iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;
bool is_active(const_iterator it) const noexcept;

void sort();
template <class Compare> void sort(Compare comp);
}

template<class InputIterator, class Allocator = allocator<iter-
value-type <InputIterator>>
    hive(InputIterator, InputIterator, Allocator = Allocator())
    -> hive<iter-value-type <InputIterator>, Allocator>;

template<class InputIterator, class Allocator = allocator<iter-
value-type <InputIterator>>
    hive(InputIterator, InputIterator, hive_limits block_limits,
Allocator = Allocator())
    -> hive<iter-value-type <InputIterator>, block_limits,
Allocator>;

template<ranges::input_range R, class Allocator =
allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, Allocator = Allocator())
    -> hive<ranges::range_value_t<R>, Allocator>;

template<ranges::input_range R, class Allocator =
allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, hive_limits block_limits, Allocator =
Allocator())
    -> hive<ranges::range_value_t<R>, block_limits, Allocator>;
}

```

hive constructors, copy, and assignment [hive.cons]

explicit hive(const Allocator&) noexcept;

1. Effects: Constructs an empty hive, using the specified allocator.
2. Complexity: Constant.

hive(hive_limits block_limits, const Allocator&);

3. Effects: Constructs an empty hive, using the specified allocator. Initializes current - limits with block_limits.
4. Complexity: Constant.

explicit hive(size_type n, const Allocator& = Allocator());
hive(size_type n, hive_limits block_limits, const Allocator& =
Allocator());

5. Preconditions: T is *Cpp17DefaultInsertable* into *this.
6. Effects: Constructs a hive with n default-inserted elements, using the specified allocator. If the second overload is called, also initializes current - limits with block_limits.
7. Complexity: Linear in n.

hive(size_type n, const T& value, const Allocator& = Allocator());
hive(size_type n, const T& value, hive_limits block_limits, const
Allocator& = Allocator());

8. Preconditions: T is *Cpp17CopyInsertable* into *this.
9. Effects: Constructs a hive with n copies of value, using the specified allocator. If the second overload is called, also initializes current - limits with block_limits.
10. Complexity: Linear in n.

template<class InputIterator>

hive(InputIterator first, InputIterator last, const Allocator& =
Allocator());

template<class InputIterator>

hive(InputIterator first, InputIterator last, hive_limits block_limits,
const Allocator& = Allocator());

11. Effects: Constructs a hive equal to the range [first, last), using the specified allocator. If the second overload is called, also initializes current - limits with block_limits.

12. Complexity: Linear in `distance(first, last)`.

```
template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, const Allocator& = Allocator());
template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& =
Allocator());
```

13. Effects: Constructs a hive object with the elements of the range `rg`, using the specified allocator. If the second overload is called, also initializes `current-limits` with `block_limits`.

14. Complexity: Linear in `ranges::distance(rg)`.

```
hive(initializer_list<T> il, const Allocator& = Allocator());
hive(initializer_list<T> il, hive_limits block_limits, const Allocator& =
Allocator());
```

16. Preconditions: `T` is *Cpp17CopyInsertable* into `*this`.

17. Effects: Constructs a hive object with the elements of `il`, using the specified allocator. If the second overload is called, also initializes `current-limits` with `block_limits`.

18. Complexity: Linear in `il.size()`.

hive capacity [`hive.capacity`]

size_type capacity() `const noexcept`;

1. Returns: The total number of elements that the hive can hold without requiring allocation of more element blocks.
2. Complexity: Constant.

void reserve(size_type n);

3. Effects: A directive that informs a hive of a planned change in size, so that it can manage the storage allocation accordingly. Does not cause reallocation of elements. Iterators to elements in `*this` remain valid. If `n <= capacity()` there are no effects.
4. Complexity: It does not change the size of the sequence and takes at most linear time in the number of reserved blocks allocated.

5. Throws: `length_error` if `n > max_size()` [numbered note](#).

6. Postconditions: `capacity() >= n` is true.

`numbered_note`) `reserve()` uses `Allocator::allocate()` which may throw an appropriate exception.

`void shrink_to_fit();`

7. Preconditions: `T` is *Cpp17MoveInsertable* into `hive`.

8. Effects: `shrink_to_fit` is a non-binding request to reduce `capacity()` to be closer to `size()`.

[Note: The request is non-binding to allow latitude for implementation-specific optimizations. - end note]

It does not increase `capacity()`, but may reduce `capacity()`. It may reallocate elements. If `capacity()` is already equal to `size()` there are no effects. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* `T`, the only effects are that `capacity()` may be reduced, element order may change and reallocation may occur.

9. Complexity: If reallocation happens, linear in the size of the sequence.

10. Throws: If reallocation to new element blocks occurs, uses `Allocator::allocate()` which may throw an appropriate exception.

11. Remarks: This operation may change the order of the elements in `*this`. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

[Note: If no reallocation happens, they remain valid. - end note]

`void trim_capacity() noexcept;`

`void trim_capacity(size_type n) noexcept;`

11. Effects: For the first overload, all reserved blocks are deallocated, and `capacity()` is reduced accordingly. For the second overload, `capacity()` will be reduced to no less than `n`.

12. Complexity: Linear in the number of reserved blocks deallocated.

13. Remarks: Does not reallocate elements and no iterators or references to elements in `*this` are invalidated.

hive modifiers [hive.modifiers]

```

template <class... Args>
    iterator emplace(Args&&... args);
iterator insert(const T& x);
iterator insert(T&& x);
void insert(size_type n, const T& x);
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<container-compatible-range <T> R>
    void insert_range(R&& rg);
void insert(initializer_list<T> il);

```

1. Complexity: Linear in the number of elements inserted. The constructor of T is called exactly once for each element inserted.
2. Remarks: For all functions, invalidates the past-the-end iterator.

```

iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);

```

3. Effects: Invalidates iterators and references to the erased element. An erase operation that erases the last element of *this also invalidates the past-the-end iterator.
4. Complexity: The number of calls to the destructor of T is the same as the number of elements erased. Additionally if any active blocks become empty of elements as a result of the function call, at worst linear in the number of element blocks.

```

void swap(hive& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value
|| allocator_traits<Allocator>::is_always_equal::value);

```

5. Effects: Exchanges the contents and capacity() of *this with that of x.
6. Complexity: Constant.

Operations [hive.operations]

1. In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in [algorithms.requirements]. The semantics of $i + n$ and $i - n$, where i is an iterator into the hive and n is an integer, are the same as those of $\text{next}(i, n)$ and $\text{prev}(i, n)$, respectively. For sort, the definitions and requirements in [alg.sorting] apply.

```
void splice(hive &x);  
void splice(hive &&x);
```

3. Preconditions: `addressof(x) != this` is true. `get_allocator() == x.get_allocator()` is true.
4. Effects: Inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements shall continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.
5. Complexity: Linear in the sum of all element blocks in `x` plus all element blocks in `*this`.
6. Throws: `length_error` if any of `x`'s active blocks are not within the bounds of `this->current_limits`.
7. Remarks: Reserved blocks in `x` are not transferred into `*this`.

```
size_type unique();  
template<class BinaryPredicate>  
size_type unique(BinaryPredicate binary_pred);
```

8. Let `binary_pred` be `equal_to< >{}` for the first overload.
9. Preconditions: `binary_pred` is an equivalence relation.
10. Effects: Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty `hive`, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is true. Invalidates only the iterators and references to the erased elements.
11. Returns: The number of elements erased.
12. Throws: Nothing unless an exception is thrown by the predicate.
13. Complexity: If `empty()` is false, exactly `size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
hive_limits block_capacity_limits() const noexcept;
```

14. Effects: Returns `current_limits`.
15. Complexity: Constant.

```
static constexpr hive_limits block_capacity_hard_limits() noexcept;
```

16. Returns: A `hive_limits` struct with the `min` and `max` members set to the implementation's hard limits.

17. Complexity: Constant.

```
void reshape(hive_limits block_limits);
```

18. Preconditions: T shall be *Cpp17MoveInsertable* into hive.

19. Effects: Assigns `block_limits` to `current_limits`. If any active blocks in `*this` are not within the bounds of `block_limits`, elements within those active blocks are reallocated to new or existing element blocks which are within the bounds. Subsequently any element blocks in `*this` which are not within the bounds of `block_limits` are deallocated. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* T, the only effects are that element order may change and reallocation may occur.

20. Complexity: Linear in the number of element blocks in `*this`. If reallocation occurs, also linear in the number of elements reallocated.

21. Throws: If reallocation to new element blocks occurs, uses `Allocator::allocate()` which may throw an appropriate exception.

22. Remarks: This operation may change the order of the elements in `*this`. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

[Note: If no reallocation happens, they remain valid. - end note]

```
iterator get_iterator(const_pointer p) noexcept;  
const_iterator get_iterator(const_pointer p) const noexcept;
```

22. Complexity: Linear in the number of active blocks in `*this`.

23. Returns: An `iterator` or `const_iterator` pointing to the same element as `p`. If `p` does not point to an element in `*this`, the past-the-end iterator is returned.

```
bool is_active(const_iterator it) const noexcept;
```

25. Complexity: Linear in the number of active blocks in `*this`.

26. Returns: `true` if and only if `it` points to an element in `*this`.

[Note: Returns `false` if the past-the-end iterator is supplied. - end note]

```
void sort();  
template <class Compare>  
void sort(Compare comp);
```

27. Preconditions: T shall be *Cpp17MoveInsertable* and *Cpp17MoveAssignable* into hive. lvalues of type T are swappable.

28. Effects: Sorts the hive according to the operator < or a Compare function object. If an exception is thrown, the order of the elements in *this is unspecified. Iterators and references to elements may be invalidated.
29. Complexity: $N \log N$ comparisons, where $N == \text{size}()$.
30. Throws: `bad_alloc` if it fails to allocate any memory necessary for the sort process. `comp` may also throw.
31. Remarks: May allocate [numbered_note](#).
[Note: Not required to be stable ([algorithm.stable]) - end note]

`numbered_note`) `sort()` uses `Allocator::allocate()` which may throw an appropriate exception.

Erasure [`hive.erase`]

```
template<class T, class Allocator, class U>
    typename hive<T, Allocator>::size_type
    erase(hive<T, Allocator>& c, const U& value);
```

1. Effects: Equivalent to:

```
return erase_if(c, [&](auto& elem) { return elem == value; });
```

```
template<class T, class Allocator, class Predicate>
    typename hive<T, Allocator>::size_type
    erase_if(hive<T, Allocator>& c, Predicate pred);
```

2. Effects: Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
return original_size - c.size();
```

VII. Acknowledgments

Matt would like to thank: Glen Fernandes and Ion Gaztanaga for restructuring advice, Robert Ramey for documentation advice, various Boost and SG14/LEWG members for support, critiques and corrections, Baptiste Wicht for teaching me how to construct decent benchmarks, Jonathan Wakely, Sean Middleditch, Jens Maurer (very nearly a co-author at this point really), Tim Song, Patrice Roy and Guy Davidson for standards-compliance advice and critiques, support, representation at meetings and bug reports, Henry Miller for getting me to clarify why the free list approach to memory location reuse is the most appropriate, Ville Voutilainen and Gašper Ažman for help with the colony/hive rename paper, Ben Craig for his critique of the tech spec, that ex-Lionhead guy for annoying me enough to force me to implement the original skipfield pattern, Jon Blow for some initial advice and Mike Acton for some influence, the community at large for giving me feedback and bug reports on the reference implementation. Also Nico Josuttis for doing such a great job in terms of explaining the general format of the structure to the committee.

VIII. Appendices

Appendix A - Basic usage examples

Using [reference implementation](#).

```
#include <iostream>
#include <numeric>
#include "plf_hive.h"

int main(int argc, char **argv)
{
    plf::hive<int> i_hive;

    // Insert 100 ints:
    for (int i = 0; i != 100; ++i)
    {
        i_hive.insert(i);
    }

    // Erase half of them:
    for (plf::hive<int>::iterator it = i_hive.begin(); it !=
i_hive.end(); ++it)
    {
        it = i_hive.erase(it);
    }

    std::cout << "Total: " << std::accumulate(i_hive.begin(),
```

```

i_hive.end(), 0) << std::endl;
std::cin.get();
return 0;
}

```

Example demonstrating pointer stability

```

#include <iostream>
#include "plf_hive.h"

int main(int argc, char **argv)
{
    plf::hive<int> i_hive;
    plf::hive<int>::iterator it;
    plf::hive<int *> p_hive;
    plf::hive<int *>::iterator p_it;

    // Insert 100 ints to i_hive and pointers to those ints to p_hive:
    for (int i = 0; i != 100; ++i)
    {
        it = i_hive.insert(i);
        p_hive.insert(&(*it));
    }

    // Erase half of the ints:
    for (it = i_hive.begin(); it != i_hive.end(); ++it)
    {
        it = i_hive.erase(it);
    }

    // Erase half of the int pointers:
    for (p_it = p_hive.begin(); p_it != p_hive.end(); ++p_it)
    {
        p_it = p_hive.erase(p_it);
    }

    // Total the remaining ints via the pointer hive (pointers will still
    be valid even after insertions and erasures):
    int total = 0;

    for (p_it = p_hive.begin(); p_it != p_hive.end(); ++p_it)
    {
        total +=>(*p_it);
    }

    std::cout << "Total: " << total << std::endl;
}

```



```
if (total == 2500)
{
    std::cout << "Pointers still valid!" << std::endl;
}

std::cin.get();
return 0;
}
```

Appendix B - Reference implementation benchmarks

Benchmark results for the colony (hive) reference implementation under GCC on an Intel Xeon E3-1241 (Haswell) are [here](#).

Old benchmark results for an earlier version of colony under MSVC 2015 update 3, on an Intel Xeon E3-1241 (Haswell) are [here](#). There is no commentary for the MSVC results.

Appendix C - Frequently Asked Questions

1. Where is it worth using a hive in place of other std:: containers?

See the final appendix for a more intensive answer to this question, however for a brief overview, as mentioned, it is worthwhile for performance reasons in situations where the order of container elements is not important and:

- a. Insertion order is unimportant
- b. Insertions and erasures to the container occur frequently in performance-critical code, *and*
- c. Links to non-erased container elements may not be invalidated by insertion or erasure.

Under these circumstances a hive will generally out-perform other std:: containers. In addition, because it never invalidates pointer references to container elements (except when the element being pointed to has been previously erased) it may make many programming tasks involving inter-relating structures in an object-oriented or modular environment much faster, and could be considered in those circumstances.

2. What are some examples of situations where a hive might improve performance?

Some ideal situations to use a hive: cellular/atomic simulation, persistent octrees/quadtrees, game entities or destructible-objects in a video game, particle physics, anywhere where objects are being created and destroyed continuously. Also, anywhere where a vector of pointers to dynamically-allocated objects or a `std::list` would typically end up being used in order to preserve pointer stability but where order is unimportant.

3. Is it similar to a deque?

A deque is reasonably dissimilar to a hive - being a double-ended queue, it requires a different internal framework. In addition, being a random-access container, having a growth factor for memory blocks in a deque is problematic (though not impossible). A deque and hive have no comparable performance characteristics except for insertion (assuming a good deque implementation). Deque erasure performance varies wildly depending on the implementation, but is generally similar to vector erasure performance. A deque invalidates pointers to subsequent container elements when erasing elements, which a hive does not, and guarantees ordered insertion.

4. What are the thread-safe guarantees?

Unlike a `std::vector`, a hive can be read from and inserted into at the same time (assuming different locations for read and write), however it cannot be iterated over and written to at the same time. If we look at a (non-concurrent implementation of) `std::vector`'s thread-safe matrix to see which basic operations can occur at the same time, it reads as follows (please note `push_back()` is the same as insertion in this regard):

std::vector	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	No
Erasure	No	No	No	No
Iteration	No	No	Yes	Yes
Read	No	No	Yes	Yes

In other words, multiple reads and iterations over iterators can happen simultaneously, but the potential reallocation and pointer/iterator invalidation caused by insertion/`push_back` and erasure means those operations cannot occur at the same time as anything else.

Hive on the other hand does not invalidate pointers/iterators to non-erased elements during insertion and erasure, resulting in the following matrix:

hive	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	Yes
Erasure	No	No	No	Mostly*
Iteration	No	No	Yes	Yes
Read	Yes	Mostly*	Yes	Yes

* Erasures will not invalidate iterators unless the iterator points to the erased element.

In other words, reads may occur at the same time as insertions and erasures (provided that the element being erased is not the element being read), multiple reads and iterations may occur at the same time, but iterations may not occur at the same time as an erasure or insertion, as either of these may change the state of the skipfield which is being iterated over, if a skipfield is used in the implementation. Note that iterators pointing to end() may be invalidated by insertion.

So, hive could be considered more inherently thread-safe than a (non-concurrent implementation of) std::vector, but still has some areas which would require mutexes or atomics to navigate in a multithreaded environment.

5. Any pitfalls to watch out for?

Because erased-element memory locations may be reused by `insert()` and `emplace()`, insertion position is essentially random unless no erasures have been made, or an equal number of erasures and insertions have been made.

6. What is hive's Abstract Data Type (ADT)?

Though I am happy to be proven wrong I suspect hives/colonies/bucket arrays are their own abstract data type. Some have suggested its ADT is of type bag, I would somewhat dispute this as it does not have typical bag functionality such as [searching based on value](#) (you can use `std::find` but it's $O(n)$) and adding this functionality would slow down other performance characteristics. [Multisets/bags](#) are also not sortable (by means other than automatically by key value). hive does not utilize key values, is sortable, and does not provide the sort of functionality frequently associated with a bag (e.g. counting the number of times a specific value occurs).

7. Why must blocks be removed from the iterative sequence when empty?

Two reasons:

- Standards compliance: if blocks aren't removed then ++ and -- iterator operations become undefined in terms of time complexity, making them non-compliant with the C++ standard.

At the moment they are $O(1)$ amortized, in the reference implementation this constitutes typically one update for both skipfield and element pointers, but two if a skipfield jump takes the iterator beyond the bounds of the current block and into the next block. But if empty blocks are allowed, there could be anywhere between 1 and `block_capacity_limits().max` empty blocks between the current element and the next. Essentially you get the same scenario as you do when iterating over a boolean skipfield. It would be possible to move these to the back of the hive as trailing blocks, or house them in a separate list or vector for future usage, but this may create performance issues if any of the blocks are not at their maximum size (see below).

- b. Performance: iterating over empty blocks is slower than them not being present, of course - but also if you have to allow for empty blocks while iterating, then you have to include a while loop in every iteration operation, which increases cache misses and code size. The strategy of removing blocks when they become empty also statistically removes (assuming randomized erasure patterns) smaller blocks from the hive before larger blocks, which has a net result of improving iteration, because with a larger block, more iterations within the block can occur before the end-of-block condition is reached and a jump to the next block (and subsequent cache miss) occurs. Lastly, pushing to the back of a hive, provided there is still space and no new block needs to be allocated, will be faster than recycling memory locations as each subsequent insertion occurs in a subsequent memory location (which is cache-friendlier) and also less computational work is necessary. If a block is removed from the iterative sequence its recyclable memory locations are also not usable, hence subsequent insertions are more likely to be pushed to the back of the hive.

8. Why not reserve all empty memory blocks for future use during erasure, or None, rather than leaving this decision undefined by the specification?

In my view the default scenario, for reasons of predictability and memory use, should be to free the memory block in most cases. But future implementations may find better strategies, somehow, and it is best not to overly constraint potential implementation. For the reasons described in the design decisions section on `erase()`, retaining the back block at least has performance and latency benefits, in the current implementation. Therefore retaining no memory blocks is non-optimal in cases where the user is not using a custom allocator. Meanwhile, retaining All memory blocks is bad for performance as many small memory blocks will be retained, which decreases iterative performance due to lower cache locality. However, one perspective is that if a scenario calls for retaining All memory blocks, this should be left to an allocator to manage. This is an open topic for discussion.

9. Why is there no default constructor for `hive_limits`?

The user must obtain the block capacity hard limits of the implementation (via `block_capacity_hard_limits()`) prior to supplying their own limits as part of a constructor or `reshape()`, so that they do not trigger undefined behavior by supplying limits which are outside of the hard limits. Hence it was not perceived by LEWG that there would be a reason for a `hive_limits` struct to ever be used with non-user-supplied values eg. zero.

10. Memory block capacities - what are they based on, how do they expand?

There are 'hard' capacity limits, 'default' capacity limits and user-defined capacity limits. Default limits (what a hive is instantiated with if user-defined capacity limits are not supplied) and user-defined limits are not allowed to go outside of an implementation's hard limits. Newly-allocated blocks also have a non-1 implementation-defined growth factor.

While implementations are free to choose their own limits and strategies here, in the reference implementation memory block sizes start from either the dynamically-defined default minimum size (8 elements, larger if the type stored is small) or an amount defined by the end user (with a minimum of 3 elements, as there is enough metadata per-block that less than 3 elements is generally a waste of memory unless the element type is extremely large).

Subsequent block sizes then increase the *total capacity* of the hive by a factor of 2 (so, 1st block 8 elements, 2nd 8 elements, 3rd 16 elements, 4th 32 elements etcetera) until the current maximum block size is reached. The default maximum block size in the reference implementation is 255 (if the type sizeof is < 10 bytes) or 8192, based on multiple benchmark comparisons between different maximum block capacities, with different sized types. For larger-than-10-byte types the skipfield bitdepth is (at least) 16 so the maximum capacity 'hard' limit would be 65535 elements in that context, for < 10-byte types the skipfield bitdepth is (at least) 8, making the maximum capacity hard limit 255.

11. What are user-defined memory block minimum and maximum capacities good for?

See the summary in paper P2857R0 which goes into this in detail.

12. Why are hive_limits specified in constructors and not relegated to a secondary function?

- a. They have always been required in range/fill constructors for the obvious reason that otherwise the user must construct, call reshape and then call range-assign/insert. This is obviously slower and more cumbersome for use.
- b. They were originally not in the default constructors due to creating ambiguity with the fill constructors, but users have asked for this since 2016. One reason for this is consistency. Another is usage with non-movable/copyable types, which cannot be used with reshape(). The guarantees of reshape have to be specified in a concrete way, so it must be able to reallocate elements when the existing blocks do not fit within the user-supplied range, and throw when it cannot do so (either due to lack of memory or some other problem). It cannot be respecified for non-movable/copyable types. The non-noexcept status of this function also caused problems for some.
- c. In 2020 the issue was discussed [in SG14](#) and Jens Maurer suggested using an external struct to make the calls unambiguous (link [here](#)), and this has been the ongoing solution. This meets the needs of those using non-movable/copyable types and is unambiguous in terms of specification.

- d. Lastly, block capacity limits are a first-order feature of the container, and something which users have repeatedly thanked me for. They are needed & will not be removed. As a side-note, it is of annoyance to many developers that similar functionality was never specified for deque, as this has led to all of the major deque implementations being unusable for large sizeof types.

13. Can a hive be used with SIMD instructions?

No and yes. Yes if you're careful, no if you're not.

On platforms which support scatter and gather operations via hardware (e.g. AVX512) you can use hive with SIMD as much as you want, using gather to load elements from disparate or sequential locations, directly into a SIMD register, in parallel. Then use scatter to push the post-SIMD-process values elsewhere after. On platforms which do not support this in hardware, you would need to manually implement a scalar gather-and-scatter operation which may be significantly slower.

In situations where gather and scatter operations are too expensive, which require elements to be contiguous in memory for SIMD processing, this is more complicated. When you have a bunch of erasures in a hive, there's no guarantee that your objects will be contiguous in memory, even though they are sequential during iteration. Some of them may also be in different memory blocks to each other. In these situations if you want to use SIMD with hive, you must do the following:

- Set your minimum and maximum group sizes to multiples of the width of your target processor's SIMD instruction size. If it supports 8 elements at once, set the group sizes to multiples of 8.
- Either never erase from the hive, or:
 1. Shrink-to-fit after you erase (will invalidate all pointers to elements within the hive).
 2. Only erase from the back or front of the hive, and only erase elements in multiples of the width of your SIMD instruction e.g. 8 consecutive elements at once. This will ensure that the end-of-memory-block boundaries line up with the width of the SIMD instruction, provided you've set your min/max block sizes as above.

Generally if you want to use SIMD without gather/scatter, it's probably preferable to use a vector or an array.

14. Why were container operators ==, != and <=> removed?

Since this is a container where insertion position is unspecified, situations such as the following may occur:

```
hive<int> t = {1, 2, 3, 4, 5}, t2 = {6, 1, 2, 3, 4};
```

```
t2.erase(t2.begin());  
t2.insert(5);
```

In this case it is implementation-defined as to whether or not `t == t2`, if the `==` operator is order-sensitive.

If the `==` operator is order-insensitive, there is only one reasonable way to compare the two containers, which is with `is_permutation`. `is_permutation` has a worst-case time complexity of $O(n^2)$, which, while in keeping with how other unordered containers are implemented, was considered to be out of place for `hive`, which is a container where performance and consistent latency are a focus and most operations are $O(1)$ as a result. While there are order-insensitive comparison operations which can be done in $O(n \log n)$ time, these allocate, which again was considered inappropriate for a `==` operator. Those operations may become the subject of a future paper.

In light of all of this the bulk of SG14 and LEWG considered it more appropriate to remove the `==`, `!=` and `<=>` operators entirely, as these were unlikely to be used significantly with `hive` anyway. This gives the user the option of using `is_permutation` if they want an order-insensitive comparison, or `std::equal` if they want an order-sensitive comparison. In either case, this removes ambiguity about what kind of operation they are expecting, and the time complexity associated with that operation.

15. What functions can potentially stop a hive from being sort()'ed?

`insert`, `emplace`, `reshape`, `splice` and `operator =` (where `*this` is destination).

16. Why was `memory()` removed?

This was a convenience function to allow programmers to find current container memory usage without using a debugger or profiler, however this was considered out of keeping with current standard practice ie. `unordered_map` also uses a lot of additional memory but we don't provide such a function. In addition, the context where it would've been useful in realtime ie. determining whether or not it's worth calling `trim_capacity()`, is better approached by comparing `size()` to `capacity()` (although this is not foolproof either since this doesn't tell us anything about whether the empty capacity is in empty blocks or between elements).

17. Why was the `Priority` template parameter removed?

This was a hint to the implementation to prioritize for lowered memory usage or performance specifically. In the reference implementation this told the container which skipfield type to use (smaller types limited block sizes due to the constraints of the jump-counting skipfield pattern). In other implementations this could've taken the form of using a bitfield with jump-counting

information pushed into the erased element memory space, for the memory usage priority. However, prior to a particular LEWG meeting there had not been sufficient benchmarking and memory testing done on this - all benchmarking had been done at an earlier time without checking memory usage.

When more thorough benchmarking including memory measurements were done it was found that the vast bulk of unnecessary memory usage came from erased elements in hive when an element memory block is not yet empty (and therefore freed to the OS or retained depending on implementation), rather than the skipfield type itself. This meant that assuming a randomised erasure pattern, smaller block capacities had far more to do with how much memory was wasted than the skipfield type, as they were more likely to be freed than larger ones. And block capacities could already be specified by the user. Further, the better performance in some benchmarks was primarily related to this fact - reusing erased element memory space in existing blocks was much faster than having to deallocate/reserve blocks and subsequently allocate/unreserve new blocks.

The only caveat to this was when using low sizeof types such as scalars, where the additional memory from the skipfield (proportional to the type sizeof) was significant, and this use-case can be worked around at compile-time by switching to a smaller skipfield type (or bitfield as described) based on sizeof(type), either using concepts and overloads or another mechanism. I personally think the priority parameter would've also been useful for a number of other compile-time decision processes, such as deciding what block retention strategy to use when erasing and a block becomes empty of elements. Also, having a priority tag gave the ability to specify new priority values in future as part of the standard, potentially allowing for new and better changes without breaking ABI in implementations.

18. Why does a bidirectional container have iterator operators $>$, $<$, $>=$, $<=$ and $<=>$?

These are useful for several reasons:

- a. It can also be used in situations where the user is looping over data and has a non-1 addition/subtraction to the iterator per-cycle (or a potential non-1 addition), and so hence having a for-loop end condition of `!= end()` or similar would not necessarily work.
- b. It is be used by the `distance()` implementation to determine whether `first > last`, and correctly calculate distance without crashing.
- c. Because hive insert location is unspecified, if you have a specific range of elements which you've calculated the distance between, iterator ops `</>`, `<=`, `>=`, `<=>` are the only way to determine whether an element you just inserted is within that range. Likewise if external objects/entities are removing elements from a hive via stored pointers/iterators, those ops are the only way to determine if the element was within that range.

19. Why is the time complexity of updating the erased-element-skipping mechanism not factored into time complexity for erase and insertion operations?

Primarily to allow for implementation improvement. While the reference implementation uses a mechanism that is $O(1)$, it is possible that someone in future could come up with a mechanism that was not $O(1)$, but still out-performed the reference implementation's mechanism substantially and was never slower. In that case we would want to switch to that mechanism. Further, since the effects of time complexity on implementation performance are both constrained by hardware and software, it's relevance is situation-dependent. But it's relevance to, for example, reallocation of elements is more reliably understood in this context, as this can be calculated based on `sizeof(value_type)`, whereas the erased-element skipping mechanism could take many forms.

The same logic applies for the erased-element recording mechanism.

20. Why is the insertion time complexity for singular insert/emplace $O(1)$ amortized as opposed to $O(1)$?

Two reasons, one is that a new block may have to be allocated or transferred from the reserved blocks if all active blocks are full. Second reason is that (at least in the current reference implementation at time of writing) in the event of a block allocation or transfer there is an update of block numbers which may occur if the last current active block has a group number == `std::numeric_limits<size_type>::max()`. The occurrence of this event is 1 in every `std::numeric_limits<size_type>::max()` block allocations/transfers. It updates the group numbers in every active block. The number of active blocks at this point could be small or large.

In addition, if a hive were implemented as a vector of pointers to groups, rather than a linked list of groups, this would also necessitate amortized time complexity as when the vector became full, all group pointers would need to be reallocated to a new memory block.

Hence $O(1)$ amortized.

Appendix D - Specific responses to previous committee feedback

1. Naming

See paper number 2332R0.

2. "Unordered and no associative lookup, so this only supports use cases where you're going to do something to every element."

As noted the container was originally designed for highly object-oriented situations where you have many elements in different containers linking to many other elements in other containers. This linking can be done with pointers or iterators in hive (insert returns an iterator which can be dereferenced to get a pointer, pointers can be converted into iterators with the supplied functions (for erase etc)) and because pointers/iterators stay stable regardless of insertion/erasure, this usage is unproblematic. You could say the pointer is equivalent to a key in this case (but without the overhead). That is the first access pattern, the second is straight iteration over the container, as you say. Secondly, the container does have (typically better than $O(n)$) advance/next/prev implementations, so multiple elements can be skipped.

3. **"Prove this is not an allocator"**

I'm not really sure how to answer this, as I don't see the resemblance, unless you count maps, vectors etc as being allocators also. The only aspect of it which resembles what an allocator might do, is the memory re-use mechanism. It would be impossible for an allocator to perform a similar function while still allowing the container to iterate over the data linearly in memory, preserving locality, in the manner described in this document.

4. **"If this is for games, won't game devs just write their own versions for specific types in order to get a 1% speed increase anyway?"**

This is true for many/most AAA game companies who are on the bleeding edge, as well as some of the more hardcore indie developers, but they also do this for vector etc, so they aren't the target audience of std:: for the most part; sub-AAA game companies are more likely to use third party/pre-existing tools. As mentioned earlier, this structure (bucket-array-like) crops up in [many, many fields](#), not just game dev. So the target audience is probably everyone other than AAA gaming, but even then, it facilitates communication across fields and companies as to this type of container, giving it a standardized name and understanding.

5. **"Is there active research in this problem space? Is it likely to change in future?"**

The only current analysis has been around the question of whether it's possible for this specification to fail to allow for a better implementation in future. This is unlikely given the container's requirements and how this impacts on implementation. Bucket arrays have been around since the 1990s, there's been no significant innovation in them until now. I've been researching/working on hive since early 2015, and while I can't say for sure that a better implementation might not be possible, I am confident that no change should be necessary to the specification to allow for future implementations, if it is done correctly. This's in part because of the [C++ container requirements and how these constrain implementation](#).

The requirement of allowing no reallocations upon insertion or erasure, truncates possible implementation strategies significantly. Memory blocks have to be independently allocated so that they can be removed (when empty) without triggering reallocation of subsequent elements. There's limited numbers of ways to do that and keep track of the memory blocks at the same time. Erased element locations must be recorded (for future re-use by insertion) in a way that doesn't create allocations upon erasure, and there's limited numbers of ways to do this also. Multiple consecutive erased elements have to be skipped in $O(1)$ time in order for the iterator to meet the C++ iterator $O(1)$ function requirement, and again there's limits to how many ways you can do that. That covers the three core aspects upon which this specification is based. See Design Decisions for the various ways these aspects can be designed.

The time complexity of updates to whatever erased-element skipping mechanism is used should, I think, be left implementation-defined, as defining time complexity may obviate better solutions which are faster but are not necessarily $O(1)$. These updates would likely occur during erasure, insertion, splicing and container copying.

6. We have (full container) splice, unique and sort, like in `std::list`, but not merge?

With splice and unique we can retain the guarantee that pointers to non-erased elements stay valid (sort does not guarantee this for `hive`), but with merge we cannot, as the function requires an interleaving of elements, which is impossible to accomplish without invalidating pointers, unless the elements are allocated individually. Such is not the case in `hive`, hence including merge might confuse users as to why it doesn't share the same property as its implementation in `std::list`. `std::sort` however is known to invalidate pointers when used with vectors and deques, so sort as a member function doesn't not necessarily have that kind of association (retaining pointer validity).

7. "Why not support `push_back` and `push_front`?"

- a. Ordered insertion would create performance penalties due to not reusing previously-erased element locations, which in turn increases the number of block allocations necessary and reduces iteration speed due to wider gaps between active elements and the resultant reduced cache locality. This negates the performance benefits of using this container.
- b. Newcomers will get confused and use `push_back` instead of `insert`, because they will assume this is faster based on their experience of other containers, and the function call itself may actually be faster in some circumstances. But it will also inhibit performance for the reasons above. Further, explaining how the container works and operates has proved to be difficult even with C++ committee members, so begin able to explain it adequately to novices such that they avoid this pitfall is in-no-way guaranteed.
- c. It should be unambiguous as to its interface and how it works, and what guarantees are in place. Making insertion guarantees straightforward is key to performant usage. Having fewer constraints is also important for allowing future, potentially-faster, implementation.

- d. Supporting `push_back` and `push_front` introduces other performance disadvantages in addition to those mentioned above. As one example if you support `push_front` you have to maintain another variable which records the point at the beginning of the container beyond which nothing has yet been inserted (usually but not always `begin()` minus 1) and be cognisant of it in your insert and erase functions. Then you also have to check in all insert/assign functions whether or not there is empty space at the front of the container prior to `begin()` which can be used prior to creating a new block.
- e. There are other, better containers for ordered insertion, even ones which support contiguous allocation and the re-use of erased element memory (eg. `plf::list`).

8. "Why not `constexpr`?" (yet)

At the present point in time, `constexpr` containers are still a new thing and some of the kinks in terms of usage may yet to be worked out. Early compiler support was not good but this is improving steadily over time. I wasn't happy with having to label each and every function as `constexpr`, as this seemed to prompt some compilers to store some results at compile time even when the container wasn't being used as `constexpr`, bloating executable size and cache use. However there seem to be movements toward labelling classes as a whole as `constexpr`, so when that comes through, it alleviates that doubt. Having said that, there are a couple of obstacles in the way of a `constexpr` hive. One is that, in the reference implementation, there is extensive use of `reinterpret_cast`'ing pointers, for three areas:

1. The per-block free-list of erased elements for re-use (by `insert/assign/emplace/etc`).
2. Allowing for aligned and overaligned element types. `get_iterator` and `is_active` rely on being able to distinguish between the element type and the aligned element type, in order to function within the standard's rules (see: design decisions, `get_iterator` and `is_active`).
3. Allocating the element block and associated skipfield block in a single allocation, which increases performance.

As `reinterpret_cast` is not allowed at compile time, 1 could be worked around by creating a union between the element type and the free list struct/pair. It's possible 2 could be done in the same way, though I lack expertise here. 3 would not be possible at compile time, and the element block and skipfield blocks would have to be allocated separately. So I think it is *possible*, though it may be a lot of work.

For the moment I am happier for `std::array` and `std::vector` to be the "canaries in the coalmine" here.

9. Licensing for the reference implementation (zLib) - is this compatible with `libstdc++/libc++/MS-STL` usage?

[Yes. zLib](#) license is compatible with both [GPL3](#) and [Apache](#) licenses (libc++/MS-STL). zLib is a more permissive license than all of these, only requiring the following:

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

Please note that "product" in this instance doesn't mean 'source code', as in a library, but a program or executable. This is made clear by line 3 which clearly differentiates source distributions from products.

In addition, high-level representatives from libc++, libstdc++ and MS-STL have stated they will either use the reference or may use it as a starting point and that licensing is unproblematic (with the exception of libc++ who stated they would need to run it past LLVM legal reps). However if in any case licensing becomes problematic as the sole author of the reference implementation [I am in a position to grant use of the code](#) under other licenses as I see fit.

10. **How does hive solve the ABA problem where a given iterator/pointer points to a given element, then that element is erased, another element is inserted and re-uses the previous element's memory location? We now have invalidated iterators/pointers which point to valid elements.**

It doesn't. Detecting these cases is down to the end user, as it is in deque or vector when elements are erased. In the case of hive I would recommend the use of either a generation counter or some other kind of unique ID within the element itself. The end user can build their own "handle" wrapper around a pointer or iterator which stores a copy of this ID, then compares it against the element itself upon accessing it.

In terms of guarantees that an element has not been replaced via hive usage, replacement may occur if:

- a. Any number of erasures have occurred, and then at least one insertion has occurred.
- b. `clear()` has been called and then at least one insertion has occurred.
- c. `shrink_to_fit()`, `reshape()`, `assign()`, `unique()`, `std::erase_if()`, `std::swap()` or `swap()` have been called.

11. Asides from it already being a known container type in game development and other domains, what are good reasons to standardise Hive?

- a. "Build it and they will come" (quote from the movie "Field of Dreams", basically means, people don't always know they want something until it's there) - once this is available and people understand the advantages (performance, memory etc) they will use it. Particularly in fields where people are doing long-term compatibility/cross-platform work they are unlikely to use non-std:: containers, even if there are significant advantages, however if it's in the standard, they will.
- b. `std::list`. Developers commonly use this for situations where they need stable pointers to elements regardless of insertion/erasure ([here](#) is an example in libreoffice, line 125 - `std::list` is used extensively in 50 libreoffice source files), but it is cache-unfriendly, slow and wasteful in terms of memory. Hive is cache-friendlier, faster and uses only one `skipfield_type` (typically 16-bit) per element to maintain it's functionality, as opposed to 2-pointers per-element for `std::list`. The only advantage of `std::list` is the ordered insertion.
- c. Other languages will beat us to the punch. I know that one person is developing a hive-equivalent for Rust. Do we want to be left behind?
- d. Having this as `std::` will allow greater communication and consistency across domains and companies.

12. What are the advantages to the user being able to reserve when an allocator can mitigate some of the effects of allocating smaller blocks rather than larger ones?

The advantage an allocator has in these circumstances is actually pretty small and limited to decreasing the number of allocation calls to the OS - that's it. While it might allocate one small block contiguous to another in the order of the sequence, it also might not (and likely won't), which decreases iteration speed. Further, there is a certain amount of metadata necessary for each block (regardless of implementation), which needs to be updated etc when erasures/insertions happen. Hence, by having more blocks than you need to, you also increase the memory overhead. There is also procedural overhead associated with each block, in terms of many of the operations like `splice`, `reshape` and `reserve`, where the more blocks you have, the more operations you incur (though the cost is low).

13. This seems over-specified/too clear in terms of it's specification.

Okay, (a) first read Appendix J so you know what you're talking about. Most of the specificity comes from the type of container and C++ specifications. (b) You know what's not over-specified? Deque. You know what the MS STL version of deque does? Allocates blocks in a fixed size of 16 bytes so that anything above 8 bytes makes it a linked list. Is that good? No. Could a hive technically do the same thing? Yes, but it would be a lot harder to justify given that block sizes are explicitly expressed in terms of numbers of elements, hence an implementation would have to have a min/max block capacity hard limit of 1. There are advantages to being more specific when you get to something more complex than an array, because it enforces good practice. This is particularly important upon first releasing the standard into the world. The standard can be easily changed later on and it's requirements widened. ABI cannot (hence, 16-byte blocks in MS STL deque).

Appendix E - Typical game engine requirements

Here are some more specific requirements with regards to game engines, verified by game developers within SG14:

- a. Elements within data collections refer to elements within other data collections (through a variety of methods - indices, pointers, etc). These references must stay valid throughout the course of the game/level. Any container which causes pointer or index invalidation creates difficulties or necessitates workarounds.
- b. Order is unimportant for the most part. The majority of data is simply iterated over, transformed, referred to and utilized with no regard to order.
- c. Erasing or otherwise "deactivating" objects occurs frequently in performance-critical code. For this reason methods of erasure which create strong performance penalties are avoided.
- d. Inserting new objects in performance-critical code (during gameplay) is also common - for example, a tree drops leaves, or a player spawns in an online multiplayer game.
- e. It is not always clear in advance how many elements there will be in a container at the beginning of development, or at the beginning of a level during play. Genericized game engines in particular have to adapt to considerably different user requirements and scopes. For this reason extensible containers which can expand and contract in realtime are necessary.
- f. Due to the effects of cache on performance, memory storage which is more-or-less contiguous is preferred.
- g. Memory waste is avoided.

`std::vector` in its default state does not meet these requirements due to:

1. Poor (non-fill) single insertion performance (regardless of insertion position) due to the need for reallocation upon reaching capacity
2. Insert invalidates pointers/iterators to all elements
3. Erase invalidates pointers/iterators/indexes to all elements after the erased element

Game developers therefore either develop custom solutions for each scenario or implement workarounds for vector. The most common workarounds are most likely the following or derivatives:

1. Using a boolean flag or similar to indicate the inactivity of an object (as opposed to actually erasing from the vector). Elements flagged as inactive are skipped during iteration.

Advantages: Fast "deactivation". Easy to manage in multi-access environments.

Disadvantages: Can be slower to iterate due to branching.

2. Using a vector of data and a secondary vector of indexes. When erasing, the erasure occurs only in the vector of indexes, not the vector of data. When iterating it iterates over the vector of indexes and accesses the data from the vector of data via the remaining indexes.

Advantages: Fast iteration.

Disadvantages: Erasure still incurs some reallocation cost which can increase jitter.

3. Combining a swap-with-back-element-and-pop approach to erasure with some form of dereferenced lookup system to enable contiguous element iteration (sometimes called a [Packed array](#)).

Advantages: Iteration is at standard vector speed.

Disadvantages: Erasure will be slow if objects are large and/or non-trivially copyable, thereby making swap costs large. All link-based access to elements incur additional costs due to the dereferencing system.

Hive brings a more generic solution to these contexts. While some developers, particularly AAA developers, will almost always develop a custom solution for specific use-cases within their engine, I believe most sub-AAA and indie developers are more likely to rely on third party solutions. Regardless, standardising the container will allow for greater cross-discipline communication.

Appendix F - Time complexity requirement explanations

Insert (single): $O(1)$ amortized

One of the requirements of hive is that pointers to non-erased elements stay valid regardless of insertion/erasure within the container. For this reason the container must use multiple memory blocks. If a single memory block were used, like in a `std::vector`, reallocation of elements would occur when the container expanded (and the elements were copied to a larger memory block). Instead, hive will insert into existing memory blocks when able, and create a new memory block when all existing memory blocks are full. This keeps insertion at $O(1)$ amortized.

If hive is structured as a vector of pointers to memory blocks instead of a linked list of memory blocks, creation of a new memory block would occasionally involve expanding the vector, itself $O(n)$ in the number of blocks, but this is still within amortized limits since it is only occasional.

Insert (multiple): $O(N)$

Multiple insertions may allow an implementation to reserve suitably-sized memory blocks in advance, reducing the number of allocations necessary (whereas singular insertion would generally follow the implementation's block growth pattern, possibly allocating more than necessary). However when it comes to time complexity it has no advantages over singular insertion, is linear to the number elements inserted.

s

Erase (single): $O(1)$ amortized

Erasure is a simple matter of destructing the element in question and updating whatever data is associated with the erased-element skipping mechanism eg. the skipfield. Since we use a skipping mechanism to avoid erasures during iterator, no reallocation of subsequent elements is necessary and the process is $O(1)$. Also, when using a Low-complexity jump-counting pattern the skipfield update is also always $O(1)$.

However if the hive is implemented via a vector-of-pointers, if a block becomes empty of elements and needs to be removed from the iterative sequence, this could, depending on implementation, trigger a $O(n)$ relocation of subsequent block pointers in the vector-of-pointers (however a smart implementation will only do this occasionally, using `erase_if` - see Appendix L for more details).

Note: When a memory block becomes empty of non-erased elements it must be freed to the OS (or reserved for future insertions, depending on implementation) and removed from the hive's sequence of memory blocks. If it was not, we would end up with non- $O(1)$ iteration, since there would be no way to predict how many empty memory blocks there would be between the current memory block being iterated over, and the next memory block with non-erased (active) elements in it.

Note2: One might think it is possible to trigger a non amortized $O(1)$ behaviour by continually inserting and erasing the same element from a full-capacity hive, in the vector-of-pointers-to-blocks style scenario. This would only be the case in an outstandingly poor implementation. It is expected that any worthwhile implementation worth its weight in bits would, when end blocks become empty of elements, keep at least one block reserved after the active blocks, in that style of implementation, for that reason. This is the same reason why an automatically-shrinking vector is undesirable.

Erase (multiple): $O(N)$ amortized for non-trivially-destructible types, for trivially-destructible types between $O(1)$ and $O(N)$ amortized depending on range start/end

In this case, where the element is non-trivially destructible, the time complexity is $O(N)$, with infrequent deallocation necessary from the removal of an empty memory block as noted above. However where the elements are trivially-destructible, if the range spans an entire memory block at any point, that block and its metadata can simply be removed without doing any individual writes to its metadata or individual destruction of elements, potentially making this a $O(1)$ operation.

In addition (when dealing with trivially-destructible types) for those memory blocks where only a portion of elements are erased by the range, if no prior erasures have occurred in that memory block you may be able to erase that range in $O(1)$ time, as, for example, if you are using a skipfield there will be no need to check the skipfield within the range for previously erased elements. The reason you would need to check for previously erased elements within that portion's range is so you can update the metadata for that memory block to accurately reflect how many non-erased elements remain within the block. The non-erased element-count metadata is necessary because there is no other way to ascertain when a memory block is empty of non-erased elements, and hence needs to be removed from the hive's iteration sequence. The reasoning for why empty memory blocks must be removed is included in the `Erase(single)` section, above.

However in most cases the erase range will not perfectly match the size of all memory blocks, and with typical usage of a hive there is usually some prior erasures in most memory blocks. So, for example, when dealing with a hive of a trivially-destructible type, you might end up with a tail portion of the first memory block in the erasure range being erased in $O(N)$ time, the second and intermediary memory block being completely erased and freed in $O(1)$ time, and only a small front portion of the third and final memory block in the range being erased in $O(N)$ time. Hence the time complexity for trivially-destructible elements is between $O(1)$ and $O(N)$ depending on the start and end of the erasure range.

The amortized part occurs for the same reasons mentioned in the single-erase complexity details above - in a vector-of-pointers-to-blocks style of implementation, there may be a need to shuffle block pointers backward when a block becomes empty of elements.

std::find: $O(N)$

This relies on basic iteration so is $O(N)$.

splice: $O(1)$ in many cases, worst case $O(n)$ in the number of active and reserved blocks in `*this`

Hive only does full-container splicing, not partial-container splicing (use `range-insert` with `std::make_move_iterator` to achieve the latter, albeit with the loss of pointer validity to the moved range). When splicing, the memory blocks from the source hive are transferred to the destination hive without processing the individual elements. These blocks may either be placed at the front of the hive or the end, depending on how full the source back block is compared to the destination back block. If the destination back block is more full ie. there is less unused space in it, it is better to put it at the beginning of the source block - as otherwise this creates a larger gap to skip during iteration which in turn affects cache locality. If there are unused element memory spaces at the back of the destination container (ie. the final memory block is not full) and a skipfield is used, the skipfield nodes corresponding to those empty spaces must be altered to indicate that these are skipped elements.

In the reference implementation `splice` is only $O(n)$ if the user is using user-defined block capacity limits, and these limits differ significantly between the source hive and destination hive. However in the event that hive is implemented structurally as a vector of pointers to element blocks + metadata,

splice will always be $O(n)$ in the number of memory blocks due to the need to copy the block pointers to the destination hive.

Iterator operators ++ and --: $O(1)$ amortized

Generally the time complexity is $O(1)$, and if a skipfield is used it must allow for $O(1)$ skipping of multiple erased elements. However every so often iteration will involve a transition to the next/previous memory block in the hive's sequence of blocks, depending on whether we are doing ++ or --. At this point a read of the next/previous memory block's corresponding skipfield would be necessary, in case the front/back element(s) in that memory block are erased and hence skipped. So for every block transition, 2 reads of the skipfield are necessary instead of 1. Hence the time complexity is $O(1)$ amortized.

If skipfields are used they must be per-element-memory-block and independent of subsequent/previous memory blocks, as otherwise you end up with a vector for a skipfield, which would need a range reallocated every time a memory block was removed from the hive (see notes under Erase, above), and reallocation to a larger skipfield memory block when a hive expanded. For both of these procedures you could have thousands of skipfield nodes needing to be reallocated based on a single erasure (from within a memory block which only had one non-erased element left and hence would need to be removed from the hive). This is unacceptable latency for any field involving high timing sensitivity (all of [SG14](#)).

begin()/end(): $O(1)$

For most implementation these should generally be stored as member variables and so returning them is $O(1)$.

advance/next/prev: between $O(1)$ and $O(n)$, depending on current iterator location, distance and implementation. Average for reference implementation approximates $O(\log N)$.

The reasoning for this is similar to that of Erase(multiple), above. Complexity is dependent on state of hive, position of iterator and length of distance, but in many cases will be less than linear. It is necessary in a hive to store metadata both about the capacity of each block (for the purpose of iteration) and how many non-erased elements are present within the block (for the purpose of removing blocks from the iterative chain once they become empty). For this reason, intermediary blocks between the iterator's initial block and its final destination block (if these are not the same block, and if the initial block and final block are not immediately adjacent) can be skipped rather than iterated linearly across, by subtracting the "number of non-erased elements" metadata from distance for those blocks.

This means that the only linear time operations are any iterations within the initial block and the final block. However if either the initial or final block have no erased elements (as determined by comparing whether the block's capacity metadata and the block's "number of non-erased elements" metadata are equal), linear iteration can be skipped for that block and pointer/index math used

instead to determine distances, reducing complexity to constant time. Hence the best case for this operation is constant time, the worst is linear to the distance.

distance: between $O(1)$ and $O(n)$, depending on current iterator location, distance and implementation. Average for reference implementation approximates $O(\log N)$.

The same considerations which apply to advance, prev and next also apply to distance - intermediary blocks between iterator1 and iterator2's blocks can be skipped in constant time, if they exist. iterator1's block and iterator2's block (if these are not the same block) must be linearly iterated across using ++ unless either block has no erased elements, in which case the operation becomes pointer/index math and is reduced to constant time for that block. In addition, if iterator1's block is not the same as iterator2's block, and iterator2 is equal to end() or (end() - 1), or is the last element in that block, iterator2's block's elements can also counted from the metadata rather than iteration.

Appendix G - Original reference implementation differences and link

This proposal and its [reference implementation](#) and the [original reference implementation](#) have several key differences, one is that the original is named 'colony', for historical and userbase reasons. Other differences follow:

- Hive is C++20 only, whereas Colony is C++98/03/11/14/17/20-compatible.
- Performance/memory priority template parameter is removed (see FAQ)
- data() function removed in hive, as may cause too much implementation specificity
- reset() function removed in hive (use clear() followed by trim_capacity() instead) - while reset() is clearer and saves some instructions, this was simply to cut down on the user interface
- Colony has several static constexpr member functions which allow the user to determine what number of elements they can get in a single element block, given a set allocation amount eg. for allocators which only supply fixed amounts of memory
- No support for sort functions other than std::sort in hive. Colony supports a user-defined internal sort function defined in a macro.

Appendix H - Some user experience reports

Richard, Creative Assembly:

"I'm the lead of the Editors team at Creative Assembly, where we make tools for the Total War series of games. The last game we released was Three Kingdoms, currently doing quite well on Steam. The

main tool that I work on is the map creation editor, kind of our equivalent of Unreal Editor, so it's a big tool in terms of code size and complexity.

The way we are storing and rendering entities in the tool currently is very inefficient: essentially we have a quadtree which stores pointers to the entities, we query that quadtree to get a list of pointers to entities that are in the frustum, then we iterate through that list calling a virtual draw() function on each entity. Each part of that process is very cache-unfriendly: the quadtree itself is a cache-unfriendly structure, with nodes allocated on the heap, and the entities themselves are all over the place in memory, with a virtual function call on top.

So, I have made a new container class in which to store the renderable versions of the entities, and this class has a bunch of colonies inside, one for each type of 'renderable'. On top of this, instead of a quadtree, I now have a virtual quadtree. So each renderable contains the index of the quadtree node that it lives inside. Then, instead of asking the quadtree what entities are in the frustum, I ask the virtual quadtree for a node mask of the nodes what are in the frustum, which is just a bit mask. So when rendering, I iterate through all the renderables and just test the relevant bit of the node mask to see if the renderable is in the frustum. (Or more accurately, to see if the renderable has the potential to be in the frustum.) Nice and cache friendly.

When one adds an entity to the container, it returns a handle, which is just a pointer to the object inside one of the colonies returned as a `std::uintptr_t`. So I need this to remain valid until the object is removed, which is the other reason to use a colony."

Andrew Shuvalov, MongoDB:

"I implemented a standalone open source project for the thread liveness monitor:
<https://github.com/shuvalov-mdb/thread-liveness-monitor>. Also, I've made a video demo of the project: <https://youtu.be/uz3uENpjRfA>

The benchmarks are in the doc, and as expected the `plf::colony` was extremely fast. I do not think it's possible to replace it with any standard container without significant performance loss. Hopefully, this version will be very close to what we will put into the MongoDB codebase when this project is scheduled."

Daniel Elliot, Weta Digital:

"I'm using it as backing storage for a volumetric data structure (like `openvdb`). Its sparse so each tile is a 512^3 array of float voxels.

I thought that having colony will allow me to merge multiple grids together more efficiently as we can just splice the tiles and not copy or reallocate where the tiles dont overlap. Also adding and removing tiles will be fast. Its kind of like using an arena allocator or memory pool without having

to actually write one."

Note: this is a private project Daniel is working on, not one for Weta Digital.

Gašper Ažman, Citadel Securities:

"Internally we use it as a slab allocator for objects with very different lifetime durations where we want aggressive hot memory reuse. It lets us ensure the algorithms are correct after the fact by being able to iterate over the container and verify what's alive.

It's a great single-type memory pool, basically, and it allows iteration for debugging purposes :)

Where it falls slightly short of expectation is having to iterate/delete/insert under a lock for multithreaded operation - for those usecases we had to do something different and lock-free, but for single-threaded applications it's amazing."

Appendix I - A brief and incomplete guide for selecting the appropriate container from inside/outside the C++ standard library, based on performance characteristics, functionality and benchmark results

Guides and flowcharts I've seen online have either been performance-agnostic or incorrect. This is not a perfect guide, nor is it designed to suit all participants, but it should be largely correct in terms of its focus. Note, this guide does not cover:

- a. All known C++ containers
- b. Multithreaded usage/access patterns in any depth
- c. All scenarios
- d. The vast variety of map variants and their use-cases
- e. Examinations of technical nuance (eg. at which sizeof threshold on a given processor does a type qualify as large enough to consider not using it in a vector if there is non-back erasure?). For that reason I'm not going to qualify 'Very large' or 'large' descriptors in this guide.

These are broad strokes and can be treated as such. Specific situations with specific processors and specific access patterns may yield different results. There may be bugs or missing information. The strong insistence on arrays/vectors where-possible is to do with code simplicity, ease of debugging, and performance via cache locality. I am purposefully avoiding any discussion of the virtues/problems of C-style arrays vs `std::array` or `vector` here, for reasons of brevity. The relevance of all assumptions are subject to architecture. The benchmarks this guide is based upon are available [here](#), [here](#). Some of the map/set data is based on [google's abseil library documentation](#).

Start!

a = yes, b = no

0. Is the number of elements you're dealing with a fixed amount?
 - 0a. If so, is all you're doing either pointing to and/or iterating over elements?
 - 0aa. If so, use an array (either static or dynamically-allocated).
 - 0ab. If not, can you change your data layout or processing strategy so that pointing to and/or iterating over elements would be all you're doing?
 - 0aba. If so, do that and goto 0aa.
 - 0abb. If not, goto 1.
 - 0b. If not, is all you're doing inserting-to/erasing-from the back of the container and pointing to elements and/or iterating?
 - 0ba. If so, do you know the largest possible maximum capacity you will ever have for this container, and is the lowest possible maximum capacity not too far away from that?
 - 0baa. If so, use `vector` and `reserve()` the highest possible maximum capacity. Or use `boost::static_vector` for small amounts which can be initialized on the stack.
 - 0bab. If not, use a `vector` and `reserve()` either the lowest possible, or most common, maximum capacity. Or `boost::static_vector`.
 - 0bb. If not, can you change your data layout or processing strategy so that back insertion/erasure and pointing to elements and/or iterating would be all you're doing?
 - 0bba. If so, do that and goto 0ba.
 - 0bbb. If not, goto 1.
 1. Is the use of the container stack-like, queue-like or ring-like?
 - 1a. If stack-like, use `plf::stack`, if queue-like, use `plf::queue` (both are faster and configurable in terms of memory block sizes). If ring-like, use [ring_span](#) or [ring_span_lite](#).
 - 1b. If not, goto 2.
 2. Does each element need to be accessible via an identifier ie. key? ie. is the data associative?
 - 2a. If so, is the number of elements small and the type `sizeof` not large?
 - 2aa. If so, is the value of an element also the key?
 - 2aaa. If so, just make an array or vector of elements, and sequentially-scan to lookup elements. Benchmark vs `absl::sets` below.
 - 2aab. If not, make a vector or array of key/element structs, and sequentially-scan to lookup elements based on the key. Benchmark vs `absl::maps` below.
 - 2ab. If not, do the elements need to have an order?
 - 2aba. If so, is the value of the element also the key?
 - 2abaa. If so, can multiple keys have the same value?
 - 2abaaa. If so, use `absl::btree_multiset`.
 - 2abaab. If not, use `absl::btree_set`.
 - 2abab. If not, can multiple keys have the same value?
 - 2ababa. If so, use `absl::btree_multimap`.
 - 2ababb. If not, use `absl::btree_map`.
 - 2abb. If no order needed, is the value of the element also the key?
 - 2abba. If so, can multiple keys have the same value?
 - 2abbaa. If so, use `std::unordered_multiset` or `absl::btree_multiset`.

2abbab. If not, is pointer stability to elements necessary?
 2abbaba. If so, use `absl::node_hash_set`.
 2abbabb. If not, use `absl::flat_hash_set`.
 2abbbb. If not, can multiple keys have the same value?
 2abbbba. If so, use `std::unordered_multimap` or `absl::btree_multimap`.
 2abbbb. If not, is on-the-fly insertion and erasure common in your use case, as opposed to mostly lookups?
 2abbbbba. If so, use [robin-map](#).
 2abbbbb. If not, is pointer stability to elements necessary?
 2abbbbbba. If so, use `absl::flat_hash_map<Key, std::unique_ptr<Value>>`. Use `absl::node_hash_map` if pointer stability to keys is also necessary.
 2abbbbbbb. If not, use `absl::flat_hash_map`.
 2b. If not, goto 3.

Note: if iteration over the associative container is frequent rather than rare, try the `std::` equivalents to the `absl::` containers or [tsl::sparse_map](#). Also take a look at [this page of benchmark conclusions](#) for more definitive comparisons across more use-cases and hash map implementations.

3. Are stable pointers/iterators/references to elements which remain valid after non-back insertion/erasure required, and/or is there a need to sort non-movable/copyable elements?

3a. If so, is the order of elements important and/or is there a need to sort non-movable/copyable elements?

3aa. If so, will this container often be accessed and modified by multiple threads simultaneously?

3aaa. If so, use `forward_list` (for its lowered side-effects when erasing and inserting).

3aab. If not, do you require range-based splicing between two or more containers (as opposed to splicing of entire containers, or splicing elements to different locations within the same container)?

3aaba. If so, use `std::list`.

3aabb. If not, use `plf::list`.

3ab. If not, use `hive`.

3b. If not, goto 4.

4. Is the order of elements important?

4a. If so, are you almost entirely inserting/erasing to/from the back of the container?

4aa. If so, use `vector`, with `reserve()` if the maximum capacity is known in advance.

4ab. If not, are you mostly inserting/erasing to/from the front of the container?

4aba. If so, use `deque`.

4abb. If not, is insertion/erasure to/from the middle of the container frequent when compared to iteration or back erasure/insertion?

4abba. If so, is it mostly erasures rather than insertions, and can the processing of multiple erasures be delayed until a later point in processing, eg. the end of a frame in a video game?

4abbaa. If so, try the `vector erase_if` pairing approach listed at the bottom of this guide, and benchmark against `plf::list` to see which one performs best. Use `deque` with the `erase_if` pairing if the number of elements is very large.

4abbab. If not, goto 3aa.

4abbbb. If not, are elements large or is there a very large number of elements?

4abbbba. If so, benchmark `vector` against `plf::list`, or if there is a very large number of elements

benchmark deque against `plf::list`.

4abbbb. If not, do you often need to insert/erase to/from the front of the container?

4abbbba. If so, use deque.

4abbbbb. If not, use vector.

4b. If not, goto 5.

5. Is non-back erasure frequent compared to iteration?

5a. If so, is the non-back erasure always at the front of the container?

5aa. If so, use deque.

5ab. If not, is the type large, non-trivially copyable/movable or non-copyable/movable?

5aba. If so, use hive.

5abb. If not, is the number of elements very large?

5abba. If so, use a deque with a swap-and-pop approach (to save memory vs vector - assumes standard deque implementation of fixed block sizes) ie. when erasing, swap the element you wish to erase with the back element, then `pop_back()`. Benchmark vs hive.

5abbb. If not, use a vector with a swap-and-pop approach and benchmark vs hive.

5b. If not, goto 6.

6. Can non-back erasures be delayed until a later point in processing eg. the end of a video game frame?

6a. If so, is the type large or is the number of elements large?

6aa. If so, use hive.

6ab. If not, is consistent latency more important than lower average latency?

6aba. If so, use hive.

6abb. If not, try the `erase_if` pairing approach listed below with vector, or with deque if the number of elements is large. Benchmark this approach against hive to see which performs best.

6b. If not, use hive.

Vector `erase_if` pairing approach:

Try pairing the type with a boolean, in a vector, then marking this boolean for erasure during processing, and then use `erase_if` with the boolean to remove multiple elements at once at the designated later point in processing. Alternatively if there is a condition in the element itself which identifies it as needing to be erased, try using this directly with `erase_if` and skip the boolean pairing. If the maximum is known in advance, use vector with `reserve()`.

Appendix J - Hive constraints summary

This is a summary of information already contained within P0447.

Constraints forced by the C++ Standard:

1. Iterator operations must be $O(1)$ amortized (iterator.requirements.general), forces:

- Non-boolean skipfield (unlike real-world implementations of bucket-lists etc) ie. jump-counting skipfield (only known possibility at present point in time) or better. For a boolean skipfield there is an undefined number of branching statements (ie. erased elements to check for and skip over) for each $++/--$ operation, worst case linear in capacity. See Design Decisions, 'A method of skipping multiple erased elements in $O(1)$ time during iteration'.
 - this, in turn enables faster, non-branching iteration with more consistent latency, and faster range-erasure/range-insertion.
- Removal of element blocks from iterative sequence once they become empty of erased elements, otherwise skipping over an undefined number of empty blocks may occur during $++/--$ operations, making those operations (in worst case) linear in the total number of blocks in terms of time complexity.
 - this, in turn, forces storage of block metadata detailing either (a) number of non-erased elements in block or (b) number of erased elements in block, to be matched with block capacity to determine block emptiness. Knowledge of each block's capacity is also necessary and may need to be recorded for each block.

2. No exceptions allowed upon erase (container.rev.reqmts), forces:

- Free-list of erased elements or free-list of runs of erased elements as the erased-element-location-recording mechanism, as opposed to a stack/vector/etc of pointers to erased element locations or similar, as the latter creates occasional allocations upon erase.

Constraints forced by container type (most real-world implementations of bucket-lists etc):

1. Stable element memory locations (reasons for which described in the introduction to P0447), forces:

- no reallocation upon insert or erase, and for most other operations.
- either linked-list/tree of individually-allocated elements, linked list of element blocks + skipfields, or container of pointers to element blocks + skipfields (only known possibilities) meet this requirement without strong memory use - see Design Decisions section '1. Collection of memory blocks + metadata', and Appendix E for more info.

2. High-speed/predictable-latency iteration, forces:

- element blocks + skipfields as opposed to linked-list/tree of individually-allocated elements, due to better memory locality.
- removal of element block from iterative sequence once empty of erased elements, otherwise skipping over potentially many empty blocks during single $++$.
 - this, in turn, forces storage of block metadata detailing either (a) number of non-erased elements in block or (b) number of erased elements in block, to be matched with block

capacity to determine block emptiness. Knowledge of each block's capacity is also necessary and may need to be recorded for each block.

3. High-speed/predictable-latency insert, forces:

- element blocks + skipfields as opposed to linked-list/tree of elements due to lower number of allocations.
- per-element-block skipfields rather than global skipfields. A global one will have vector characteristics, creates $O(n)$ reallocation when insert triggers creation of new memory block.

4. High-speed/predictable-latency erase, forces:

- element blocks + skipfields as opposed to linked-list/tree of elements due to lower number of deallocations.
- per-element-block skipfields rather than global skipfields. A global one will have vector characteristics, creates $O(n)$ erase when removing an element block from the iterative sequence, when that element block becomes empty of non-erased elements.
- per-element-block free lists of erased elements as opposed to a global free list. A global one would mean that, upon removal of a block from the iterative sequence (when it becomes empty of non-erased elements), hive would have to traverse the entirety of the global free list ($O(n)$ in the number of erased elements within the hive) in order to remove all the free list entries from that block.

Constraints forced by colony/hive (compared to some bucket array-type implementations):

1. Higher-speed iteration/insert and less memory usage, forces:

- Reuse of erased element locations rather than simply erasing till an element block becomes empty, then removing the block (not re-using locations lowers element locality, increases number of block deallocations/allocations and memory usage). Some bucket-array-like structures also do this. See Design Decisions, 'Erased-element location recording mechanism'.
 - this, in turn, forces random positions for insertions. Bucket-array structures without reuse can guarantee insertion at back of container, at the cost of iteration speed and memory.
- Growth factor for element block capacities (up to limit) unless user specifies min/max block capacity limits as equal. This reduces number of allocations and increases element locality compared to static block capacities, when user does not know their max amount of elements in advance of construction.
 - this, in turn, forces storage of metadata on capacity for each memory block.
 - It also provides the facility for a user to specify their own block limits with minimal change to an implementation.

2. >, <, >=, <= and <=> iterator operators primarily to allow for ease-of-use with loops with greater-than-1 iterator incrementations, forces:

- Storage of element block number (ordering of block in iterative sequence) metadata. In present implementation this number only needs to be updated once in every `std::numeric_limits<size_type>::max()` block allocations, and occasionally during splice.
- this, in turn, enables the `std::distance(first, last)` overload for hive to calculate negative distances between iterators rather than generating undefined behavior when first is later in the iterative sequence than last.

So in order to serve the requirements of high performance, stable memory locations, and the C++ standard, a standard library implementation of this type of container is very constrained as to how it can be developed. Ways of meeting those constraints which deviate from reference implementation are detailed in this paper under Design Decisions and in Appendix L.

Appendix K - Prior art links

In addition to the below, I have written a supporting paper which attempts to delineate prevalence of use within industry - with the results being about 61% of respondents using something like it (see [P3011](#)).

Sean Middleditch talks about 'arrays with holes' on his old blog, which is similar but he uses id lookup instead of pointers. There is some code: [link](#)

Jonathan Blow talking about bucket arrays (note, he says "I call it a bucket array" which might connote that it's his concept depending on your interpretation, but it is actually just an indication that there are many names for this sort of thing): [link](#)

A github example (no iteration, lookup-only based on entity id as far as I can tell): [link](#)

This guy describes free-listing and holes in 'deletion strategies': [link](#)

Similar concept, static array with 64-bit integer as bit-field for skipping: [link](#)

Going over old colony emails I also found someone whose company had implemented something like the above but with atomic 64-bit integers for boolean (bitfield) skipfields and multi-blocks for multithreaded use. Note: The same space-saving as a bitfield is possible with hive by using a bitfield to indicate erased elements then storing the jump-counting data in the erased element memory space. Bitfields by themselves are not possible due to C++ standards for iteration as noted. See Design Decisions: "2. A method of skipping erased elements in O(1) time during iteration".

In my time promoting this, I initially thought it was a new concept, but gradually, and particularly after doing the cppcon talk, more and more people kept coming forward saying, yes, we do this, but with X difference- and I realised it was already a common pattern, but that this way of doing it is more performant, and the only way of getting it to meet the standard requirements, as noted.

Pool allocators etc are often constructed similarly to hives, at least in terms of using free lists and multi memory blocks. However they are not useful if you have large amounts of elements, all or most of which require bulk processing for repeated timeframes, because an allocator doesn't provide iteration, and manually iterating via say, a container of pointers to objects in a pool, has the same performance problem as linked lists- plus the memory waste.

Appendix L - Further info on non-reference-implementation designs

I will give the summary first, then show in detail, how we get there and why some approaches aren't possible. Note when I talk about vectors below I'm talking about a simplified custom vector not `std::vector`.

The non-reference-implementation approach, in summary

Like in the reference implementation, there are structs (referred to herein as 'groups') containing both an element array + skipfield array + array metadata such as size, capacity etc. Each group has it's own erased-element free list just like the reference implementation.

The hive contains a vector of pointers to groups (referred to herein as a 'group-vector'). The group-vector contains 2 extra pointers, one at the front of the active group pointers and one at the back of the active group pointers, each of which has it's own location in memory as it's value (these are referred to herein as the front and back pointers).

Each allocated group also contains a reverse-lookup pointer in it's metadata which points back to the pointer pointing at it in the group-vector. While this is used in other operations it is also used by iterator operators `>/</>=` etc to determine whether the group that iterator1 is pointing to is later than the group that iterator2 is pointer to, in the iterative sequence.

An iterator maintains a pointer to the group, the current element and the current skipfield location (or just an index into both the element and skipfield arrays). When it needs to transition from the end or beginning of the element array to the next or previous group, it takes the value of the reverse-lookup pointer in the current group's metadata and increments or decrements it respectively.

If the value of the memory location pointed to is NULL, it increments/decrements again till it finds a non-NULL pointer - this is the next block. If the value of the memory location pointed to is equal to the memory location, the iterator knows it has found the front or back pointer, depending on whether it was decrementing or incrementing respectively.

When a group becomes empty of non-erased elements, it is either deallocated or retained for future insertions by copying it's pointer in the group-vector to an area past the back pointer, depending on implementation. Either way it's original pointer location in the group-vector is NULL'ed.

There is a hive member counter which counts the number of NULL'ed pointers. If it reaches a implementation-specific threshold, a `erase_if` operation is called on the vector, removing all NULL pointers and consolidating the rest. Subsequently (or as part of the `erase_if` operation) the groups whose pointers have been relocated have their reverse-lookup pointer updated. The threshold prevents (a) iterator `++/--` straying too far from $O(1)$ amortized in terms of number of operations and (b) too many `erase_if` operations occurring.

Likewise for any splice operation, when source groups become part of the destination, the destination group-vector gets pointers to those groups added, and the reverse-lookup pointers in those groups get updated. All reverse-lookup pointers get updated when the vector expands and the pointers are reallocated.

To keep track of groups which currently have erased-element locations ready to be re-used by insert, we can either keep the reference implementation's intrusive-list-of-groups-with-erasures approach, or we can remove that metadata from the group and instead have a secondary vector of `size_type` which has the same capacity as the group-vector, containing a jump-counting skipfield which skips groups which do not have erasures. When one wants to insert one would do a single iteration on this skipfield, which would lead to a group with erasures. This approach replaces 2 pointers of metadata per-group with one `size_type`.

In that skipfield we maintain a record of runs of groups which do Not currently have erased element locations available for reuse, so that if there are any such groups available, a single iteration into this skipfield will take us to the index corresponding to that group in the group-vector. And if there are no such groups available, that same iteration will take us to the end of the skipfield.

If insertion determines that there are no such groups available, it can (depending on implementation) either check the hive member counter which counts the number of NULL'ed pointers, and if it's not zero, linear-scan the group-vector to find any NULL locations and reuse those to point to a new group - or it could just move the back pointer forward 1 and reuse that location to point to a new group (relying on the occasional `erase_if` operations to clean up the NULL pointer locations instead, and running `erase_if` itself if the vector has reached capacity). If the implementation has retained a pointer to an empty group past the back pointer (a group made empty by erasures) it can reuse that at this point.

Alternative strategies within the above approach

1. One can, instead of storing all block metadata with the block, store some of it with the pointer in the vector-of-pointers, or even in it's own vector (following a struct-of-arrays-style formation).
2. For example, one could store capacity and size with the pointer in the vector instead of in the group, and set capacity to 0 when a group is removed. The iterator could then use the value of capacity, instead of nulling the pointer, to indicate a vector entry they need to skip when iterating. The hive could then use the pointers themselves to form a free list of groups-with-erased-elements, instead of the approaches described above - requiring no additional memory.
3. Alternatively we could just have a separate vector-based bitfield with 1's for groups which contain erasures, and linearly-scan the bitfield to find the first group with erasures. This reduces

the memory cost down to 1 bit per group at the cost of decreased performance and greater variability in latency due to the use of branching. Note: the current specification prohibits this, as the fields which have been initially designed for favour consistent latency over improved average latency (eg. gaming).

4. Alternatively (and again the current specification prohibits this) again we could store the per-block free list of erased elements head in a separate vector, and linearly-scan for the first free list head without a 'no erasures' value (if copying current implementation that is `numeric_limits<lskipfield_type>::max()`). Again we run the risk of latency due to branching, but this consumes no additional memory to record which blocks have erasures.

How we get there, starting from the simplest approach

The simplest idea I had for that alternative (non-reference-implementation) approach was, a vector of pointers to allocated memory blocks (which is what `plf::list` uses). In terms of how iteration works, the iterator holds a pointer to the vector-of-pointers (the structure, not the vector's internal array) and an index into the vector-of-pointer's array, as well as a pointer/index to the current element. The hive itself would also store a pointer to the vector structure, allocating it dynamically, which makes swapping/moving non-problematic in terms of keeping valid iterators (if the hive stores the vector as a member, the iterator pointers to the vectors get invalidated on swap/move).

When the end of a block is reached by the iterator, if it hasn't hit `end()` you add 1 to the vector-of-pointers index in the iterator and continue on to the next block. Since the iterator uses indexes, reallocation of the vector array upon expansion of the vector-of-pointers doesn't become a problem. However it *is* a problem when a block prior to the current block that the iterator is pointing at, becomes empty of elements and has to be removed from the iterative sequence. If the pointer-to-that-block gets erased from the vector-of-pointers, that will cause subsequent pointers to be relocated backward by one, which in turn will make iterators pointing to elements after that block invalid (because the relocation invalidates the stored block indexes in the iterators).

Substituting a swap-and-pop between the erasable pointer and the back pointer of the vector-of-pointers, instead of erasing/relocating, doesn't solve this problem, as this produces unintuitive iteration results when an iterator lies between the back block and the block being erased (suddenly there is a bunch of elements behind it instead of in front, so forward iteration will miss those), and it also invalidates iterators pointing to elements in the (previously) back block.

So at this point there are two approaches, A & B.

Approach A

Here we have to think it terms of what's efficient, not what necessarily lowers time complexity requirements. Basically instead of erasing pointers to the erased blocks from the vector, we mark them as NULL and the iterator, when it passes to the next block, skips over the NULL pointers. This is the opposite of what we try to do with the current approach in the reference implementation (remove blocks from the iterative linked-list-of-blocks sequence) because with the current approach,

those blocks represent a latency issue via following pointers to destinations which may not be within the cache. With a vector approach however, it makes no difference to latency because the next pointer in the vector chunk already exists in the cache in, at a guess, 99% of cases. You could, potentially, get a bad result when using a CPU with poor branch-prediction-recovery performance like core2's (because this approach introduces a branching loop), when you have a close-to-50/50 random distribution of NULL blocks and actual pointers. But since blocks are generally going to be many factors fewer than elements within those blocks, this is not likely to be a major performance hit like a boolean skipfield over elements would be, even in that case.

In terms of re-using those NULL-marked pointers, we can't do a free-list of pointers because then during iteration we would have no idea which pointer was a pointer to a block and which a pointer to another free-list item - so instead we simply have a `size_type` counter in the hive metadata which counts the number of erased pointers currently in the vector-of-pointers. When we reach the capacity of existing element blocks and need to create a new block upon insert, we check the counter - if it's not 0 (ie. time to create new a block pointer at the back of the vector), scan manually along the vector-of-pointers until you hit a NULL and re-use that (same logic as above as to why this isn't a latency/performance issue) and decrement the 'erased pointer' counter.

Since insertion location is unspecified for hive, inserting a block randomly into the middle of the iterative sequence causes no fundamental issues, is the same as re-using an erased element location during insertion.

Approach B

If one is concerned about strict time complexity, and less concerned about real world effects of that time complexity, you can basically have a jump-counting skipfield for the vector-of-pointers (secondary vector of `size_type` with a low-complexity jump-counting skipfield).

This means (a) iterators can skip over pointers to erased blocks in $O(1)$ time and (b) the memory locations of the pointers to erased blocks can be used to form a free list of reusable pointers. So this eliminates both of the non- $O(1)$ aspects of Approach A, though whether or not this is faster in-practice is down to benchmarking.

Metadata and probable advantages of either approach

I've left out block metadata (size, capacity, erased element free list head, etc) in the descriptions above to simplify explanation, but for approach A we would probably want block metadata as part of a struct which the vector-of-pointers is pointing to (struct contains the element block too), so that the non- $O(1)$ linear scans over the vector-of-pointers are as fast as possible.

For approach B we would probably want the vector-of-pointers to actually be a vector-of-struct-metadata, with the pointer to the element block being one of the pieces of metadata. We could also do a 'struct of arrays' approach instead, depending on the performance result.

It's impossible to gauge whether approach A or B would be faster than the reference implementation without benchmarking (using probable usage patterns as benchmarks), but both have an advantage over the reference implementation of reducing metadata memory use.

Both approaches eliminate the need for the 'block number' piece of metadata since we get that from the vector-of-pointers for free. They also eliminate the need for prev-block/next-block pointers, though this is offset by the need for the vector of pointers in approach A and for approach B the secondary skipfield - but still a reduction of 2 pointers to 1 pointer/size_type.

The intrusive-list-of-blocks-with-element-erasures from the reference implementation could in this approach be replaced with a [high-complexity_jump-counting_skipfield](#) (an additional vector of size_type) which, instead of denoting runs of erased blocks, denotes runs of blocks with no element erasures (includes erased blocks), ie. with no prior erasures, iterating over this skipfield would jump directly to the end of the skipfield on the first iteration. This further reduces the memory use per-block of recording down from 2 pointers in the reference implementation, to 1 size_type.

Alternatively if we go the vector-of-metadata-structs route, and don't mind doing a non-O(1) linear scan upon insert, we can linear-scan the erased-element-free-list-head metadata of each block to find blocks with erasures, subsequently eliminating additional memory use for recording blocks-with-erasures entirely. This approach would be benefited by splitting the vector-of-metadata-structs into a struct-of-vectors for each metadata item.

However, 'Splice()'

All of this forgets splice, which requires that iterators to the source and destination hive's elements not be invalidated when the source hive's elements are transferred to the destination hive.

If we take a vector-of-pointers/vector-of-metadata approach, and our iterators use indexes into that vector, those indexes will be invalidated during splice, as the source vector's contents must be transferred into the destination vector. Moreover the pointer-to-vector which the iterator must hold in order to transition between blocks, would also be invalidated during splice for iterators pointing to source elements - which means that just swapping from vectors to deques and using pointers instead of indexes within the iterators, won't help.

The solution is unintuitive but works. The iterator becomes much the same as the reference implementation: either 3 pointers, one to the block+metadata struct, one to the element and one to the skipfield, or 1 pointer (to the struct) and one index (into the element and skipfield blocks respectively). We add a "reverse-lookup" pointer (or index, but we'll say pointer for the remainder of this text) to the element block's metadata which points back at the pointer pointing to it in the vector. When the iterator needs to transition blocks it follows the pointer out to the vector and increments or decrements as necessary. When a splice occurs, it alters the reverse-lookup pointers in each of the source's blocks such that they point to the correct pointers in the destination's vector.

Neither move nor swap are required to update these pointers, since those operations will simply swap the members of the vector including the pointer to the dynamically-allocated internal array, and neither the block metadata nor the iterators contain pointers to the vector itself. As such we no

longer need to dynamically-allocate the vector-of-pointers in the hive and can just have it as a member.

The solution does not entirely rule out Approach B (vector of metadata structs) in the above sections, but simply means that the reverse-lookup pointer *must* be stored with the element block, while other metadata may be stored either with the element block, or in the vector, or in separate vectors (ie. in a struct-of-arrays style), as desired or is determined to perform well. Also:

This solution allows us to fully erase entries from the vector and relocate subsequent entries, since we're no longer relying on indexes within iterators to keep track of the block pointer location in the vector. An implementation can choose whether or not they want to consolidate the vector after a block erasure, and might want to have a maximum threshold of the number of erased entries in the vector before doing so (or possibly the number of Consecutive erased entries). This prevents the number of operations per ++/-- iterator operation from becoming too high in terms of skipping erased entries. Most importantly, keeps ++/-- iterator operation at $O(1)$ amortized (and removes any performance problems relating to poor branch-prediction-recovery, as described earlier).

The vector erase operation (erase_if if we're following a threshold approach and consolidating multiple erased entries at the same time) would go through block metadata and adjust the reverse-lookup pointer in each of them to the new correct locations. Likewise when insertion triggers an expansion of the vector, the reverse-lookup's get updated (if a deque is used instead of a vector, this last is unnecessary as no reallocation takes place upon insert).

Lastly, we need a way for the iterator to figure out that it's at the beginning or end of the vector-of-pointers respectively. While we could store a pointer to the vector itself in the block metadata as well, this is quite wasteful. A less wasteful solution is to have pointers at the beginning and end of the vector-of-pointers which have special values. Instead of wasting a pointer per-block, we waste only two pointers per hive in this way. The special value (since NULL'ing the element block pointer is already taken) can be the address of the pointer location itself, since this is unique.

iterator operators >/</>=/etc

Now we lack a group_number metadata entry but we also lack a way to obtain the group number from the vector-of-pointers, since neither the block metadata nor the iterator currently store a pointer to the vector (and the iterator can't, since the pointer might get invalidated and the iterator can't get automatically updated by the container).

But luckily we don't need to know the group number for these operations to work; we only need to know if one group is later in the sequence than the other, and since we're storing a reverse-lookup pointer to the pointer in the vector-of-pointers, when comparing to see if $it1 < it2$ we only need to check whether $it1->block_metadata->reverse_lookup$ is $< it2->block_metadata->reverse_lookup$. Simple.

If we use a deque instead of a vector we can't do the above as the pointers are not guaranteed to point to locations within the same block. So for a deque we would have to store pointers to the vector in each block in order to get block numbers, which is wasteful, and update those pointers

upon splice/move/swap. We could however remove the back and front pointers in the deque-of-pointers at that point, as the iterator could use the pointer to the vector to find the front and back of the deque, instead. But in general we probably want to use a vector instead due to the simplicity of implementation and manageability.

So, that's it. This is probably not the only approach possible when not using the reference implementation, but it works.

Additional info for supporting small types

In the reference implementation we do not accommodate very small types (ie. 1 byte) without over-aligning the type to be $\text{sizeof}(\text{skipfield_type}) * 2$ - this means they are overaligned to 2 bytes. This is in order to accommodate the doubly-linked free list of runs of erased elements ("skipblocks" in the terminology of the reference) which is expressed in pairs of prev/next indexes written to the erased element's memory location. Those indexes have to be able to address the whole of the specific element block, which means they have to be the same size as skipfield_type.

If an implementation wished to create an overload for very small types (8-bit in this case) such that there was no over-alignment of the element nor wasted memory due to this, there are 3 valid approaches that I can think of.

1. 16-element blocks + 4-bit skipfield

In this approach we reduce the skipfield bit-depth to 4-bits which in turn reduces the maximum size of element blocks to 16 elements. We can do this via a [bit-field](#) or similar. We can then continue business-as-usual and store our free list node data in the erased element memory space.

2. Maximum 256-element blocks + high-complexity jump-counting skipfield + 8-bit skipfield + free list of erased elements (instead of skipblocks)

Here we substitute the low-complexity jump-counting skipfield for the high-complexity variant, which is at worst $O(n)$ in the number of skipfield nodes in a given block, however, since our maximum block size is 256 elements (256 corresponding skipfield nodes) this caps the number of operations, effectively making it $O(1)$ (this is what has been conveyed to me). This allows us to have a free list of erased elements instead of skipblocks, since we can find out the relative position of a node within a skipblock from the node's value in the high-complexity pattern. In turn this means we only need to store a 'next' value in our free list instead of both a 'prev' and 'next' value (see "Erased Element recording mechanism" under Design Decisions for more information on why this is the case). So we can store our 8-bit 'next' value in the erased-element memory space without over-aligning the type.

3. 16-element blocks + high-complexity jump-counting skipfield + 1-bit skipfield + free list of erased elements (instead of skipblocks)

Here we take the "bitfield + jump-counting skipfield" approach described in the Design Decisions section and use it with a 4-bit high-complexity jump-counting skipfield. This allows

us to store the jump-counting value + the free list 'next' value in the erased element memory space (since both are 4-bits and the type is 8-bits).

However if we look at the memory cost of each of these approaches, only one is adequate. Assuming we take a 'vector-of-pointers' approach to implementation + a jump-counting skipfield of blocks-with-erasures (as described in the summary at the start of this appendix) + doing a few other things like allocating the element block and skipfield block in one allocation, and using the offset in capacity from the start of the element block to find the skipfield block (instead of storing a pointer to the skipfield block), we can reduce the per-block metadata down to around 40 bytes. Taking each of the approaches listed above as well as the reference implementation's approach, for 256 elements we would end up with the per-element memory cost being:

1. 16-element blocks + low-complexity jump-counting skipfield + 4-bit skipfield + free list of skipblocks

Metadata cost: $16 * 40$ bytes

Skipfield cost: $256 * 4$ bits (.5 bytes)

Additional memory cost per element: 3 bytes.

2. Maximum 256-element blocks + high-complexity jump-counting skipfield + 8-bit skipfield + free list of erased elements

Metadata cost: $1 * 40$ bytes

Skipfield cost: $256 * 1$ bytes

Additional memory cost per element: 1.16 bytes.

3. 16-element blocks + high-complexity jump-counting skipfield + 1-bit skipfield + free list of erased elements

Metadata cost: $16 * 40$ bytes

Skipfield cost: $256 * 1$ bit (.125 bytes)

Additional memory cost per element: 2.63 bytes.

4. (reference implementation, ie. overalign the type to 16-bits) Maximum 256-element blocks + low-complexity jump-counting skipfield + 8-bit skipfield + free list of skipblocks

Metadata cost: $1 * 40$ bytes

Skipfield cost: $256 * 1$ bytes

Overalignment cost: $256 * 1$ bytes

Additional memory cost per element: 2.16 bytes.

Clearly the second approach is the only viable one, if memory waste is a concern. It also allows us to allocate larger numbers of elements contiguously and reduces the number of allocations/deallocations, which improves insert/erase/iteration performance compared to the 1st and 3rd approaches with 16-element blocks.